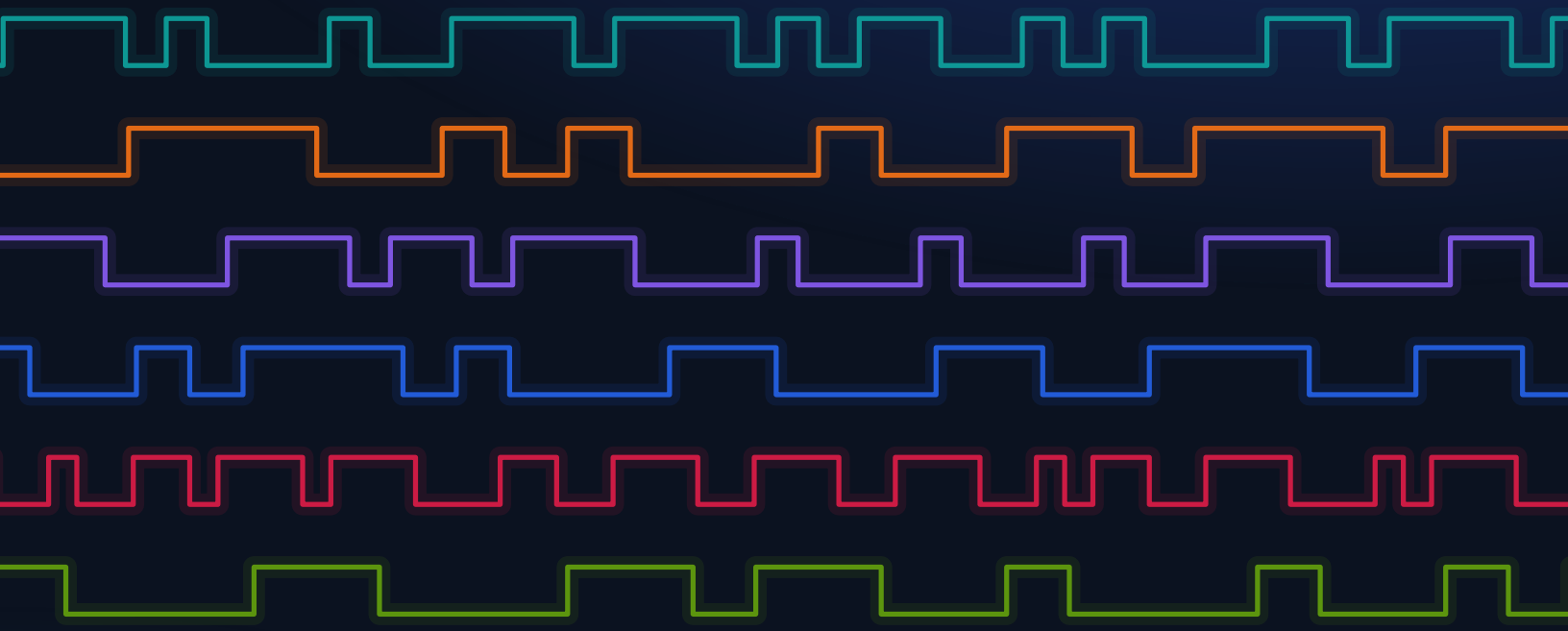
MINTS · MULTI-SCALE INTEGRATED INTELLIGENT INTERACTIVE SENSING

Wired Data Links for Environmental Sensors

How I²C, UART, SPI, USB, RS-232 and RS-485 move readings from air-quality, gas and meteorological sensors into your logger, and how to make them work in the field.

I²C · UART · SPI · USB & USB-Serial · RS-232 · RS-485



PART I · FOUNDATIONS

How each protocol works

Concise, visual introductions for CE/CS students: wiring, framing, addressing, parity, start/stop bits, baud rate, then side-by-side comparison tables.

PART II · PRACTICE

Building reliable sensor nodes

Design math (pull-ups, baud error, biasing, cable limits), Linux/Raspberry Pi specifics, Modbus RTU, data integrity, debugging and deployment.

Training guide for undergraduate & graduate researchers

Version 1.0 · October 2026

START HERE

Contents & how to use this guide

Read Part I in order. Keep Part II open on the bench when you build and debug.

PART I · FOUNDATIONS

- 1 · The language of serial links 4
- 2 · I²C 6
- 3 · UART 10
- 4 · SPI 13
- 5 · USB & USB-to-serial 16
- 6 · RS-232 19
- 7 · RS-485 21
- 8 · Similarities & differences 23

PART II · PRACTICE & ADVANCED TOPICS

- 9 · Anatomy of a sensor node 29
- 10 · Electrical ground rules 30
- 11 · I²C in practice 32
- 12 · UART in practice 35
- 13 · SPI in practice 39
- 14 · USB, USB-serial & RS-232 in practice 41
- 15 · RS-485 & Modbus RTU in practice 42
- 16 · Data integrity across the stack 46
- 17 · NMEA 0183 & SDI-12 48
- 18 · Bring-up & troubleshooting 48
- 19 · Checklist & formula sheet 50
- References 51

What you will be able to do

- Explain how each of the six links frames, clocks and addresses data, and draw its waveform.
- Say what slave addressing, parity, start/stop bits, baud rate, ACK, chip select, termination and biasing each *achieve*.
- Pick the right link for a sensor, a cable length and a node count.
- Size I²C pull-ups, check UART baud error, bias and terminate an RS-485 bus, and estimate wire time.
- Bring up sensors on a Raspberry Pi/Linux logger (i2c-tools, pyserial, spidev, Modbus RTU).
- Debug methodically with a multimeter, logic analyzer and oscilloscope.

Reading the visuals

I²C UART SPI USB RS-232 RS-485

Each protocol keeps its colour on every page. Every waveform is drawn from the actual bits it shows (e.g. address 0x76, byte 0x71), and every chart is computed from the formula given next to it. Chart cards use pinned "tooltips" to mark worked examples. Callouts: **blue** = background, **green** = good practice, **amber** = common trap, **red** = can damage hardware or corrupt data.

PART I

Foundations

01

Six ways to move bits between a sensor and a computer. Each section answers the same questions: which wires, who drives the clock, how a device is selected, how a byte is framed, and how errors are caught.

1 · Vocabulary
bit rate vs baud, synchronous vs asynchronous, duplex, single-ended vs differential, open-drain vs push-pull

2–4 · On-board buses
I²C (two wires, addresses), UART (no clock, start/stop bits, parity), SPI (fast, chip-select)

5 · USB
host-controlled bus; how a sensor shows up as /dev/ttyUSB0 or COM5

6–7 · Cable standards
RS-232 (bipolar, point-to-point) and RS-485 (differential, multi-drop, long cables)

8 · Comparison
similarities, differences, what each mechanism achieves, and how to choose

Prerequisites
binary/hex, Ohm's law, basic C or Python. Nothing else.

SECTION 1

The language of serial links

0101

Serial means one bit at a time on each data wire. Fewer wires, longer cables, simpler connectors. The price is that both ends must agree exactly on how to cut the bit stream back into bytes.

Term	Meaning	Why you care
Bit rate	Raw bits per second on the wire.	Sets how long each bit lasts: bit time = 1 / bit rate (9600 bit/s → 104.17 µs).
Baud	Symbols per second. All six links here send one bit per symbol, so baud = raw bit rate.	Useful data rate is lower: start/stop bits, addresses, ACKs and CRCs are overhead.
Synchronous	A clock wire (or embedded clock) tells the receiver when to sample. I²C, SPI.	No speed agreement needed; the master can even pause the clock.
Asynchronous	No clock wire; both ends pre-agree on the bit time. UART (and so RS-232/RS-485 links).	A start bit re-synchronises the receiver at every character.
Half / full duplex	Both directions, but one at a time (I²C, 2-wire RS-485, USB 2.0) or simultaneously (UART, SPI, RS-232).	Half-duplex needs turn-taking rules (who may talk when).
Single-ended / differential	Voltage measured against ground, or as the difference between two paired wires (RS-485, USB).	Differential rejects noise and ground shifts → long cables.
Push-pull / open-drain	Output actively drives HIGH and LOW, or can only pull LOW (a resistor makes HIGH). I²C is open-drain.	Open-drain lets many devices share a wire without short-circuits.
Bit order	UART sends LSB first; I²C sends MSB first; SPI is usually MSB first.	Reversed bit order is a classic decoding bug.
Idle state	The line level when nobody talks: UART HIGH, I²C both lines HIGH, RS-232 negative ("mark").	A transmission is recognised as a departure from idle.

Table 1. Core vocabulary used throughout the guide.

★ **A note on words**

This guide uses **master/slave**, the terms in most sensor datasheets you will read. Newer documents rename the same roles: **controller/target** (NXP I²C specification UM10204, Rev. 7, 2021), **controller/peripheral** with pins **COPI/CIPO** for SPI, and **host/device** for USB. The electrical behaviour is identical.

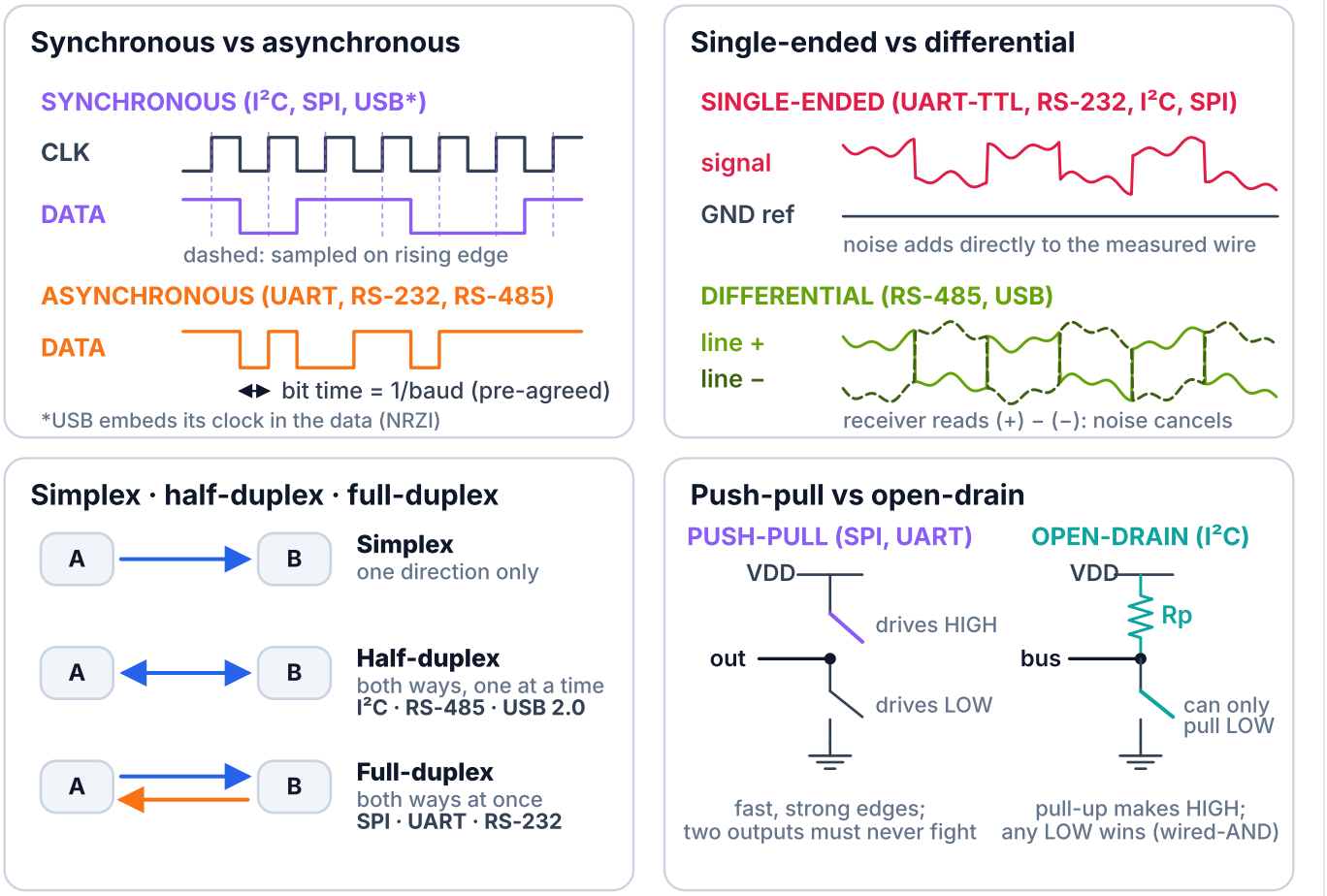


Figure 1. Four distinctions that explain most of the differences between the six protocols. Waveforms are schematic.

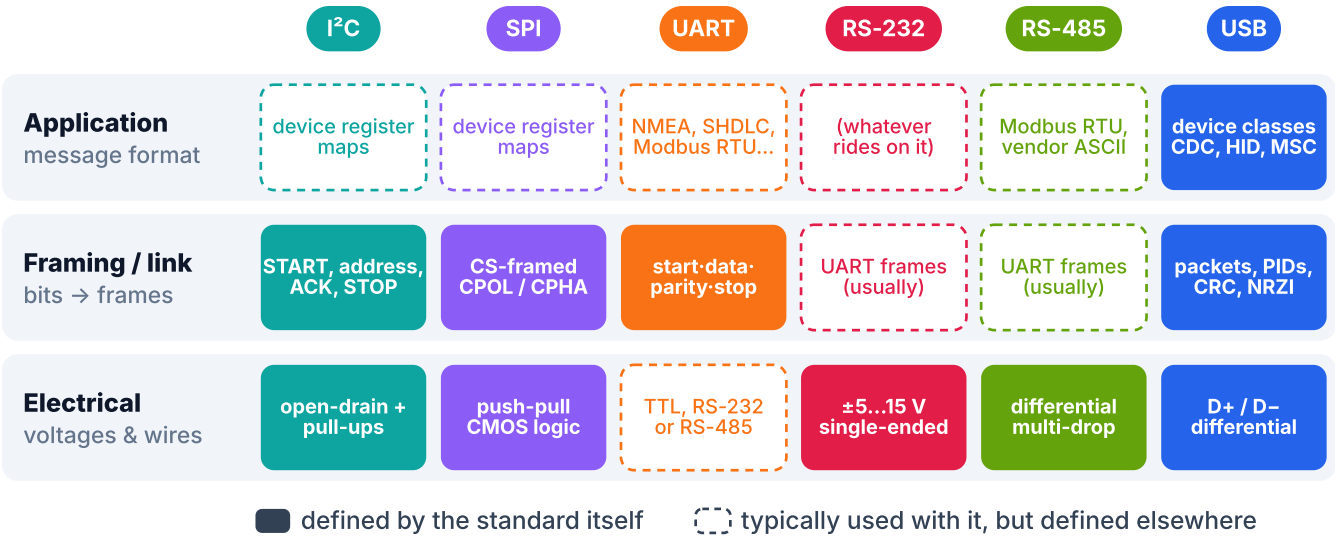


Figure 2. What each standard actually defines. UART is a framing scheme with no voltage rules; RS-232 and RS-485 are electrical standards that usually carry UART frames; USB defines everything from wires to device classes.

The single most useful idea in this guide

"Serial port" is ambiguous. A UART frame (start · data · parity · stop) can travel on a 3.3 V logic pin, on a ±10 V RS-232 cable, on an RS-485 twisted pair, or inside USB packets. The bytes and the settings (e.g. 9600 8N1) stay the same; only the electrical layer changes. Converting between them needs a **transceiver chip**, not new software.

SECTION 2

I²C

Inter-Integrated Circuit

Two shared wires connect many chips on a board. Every device has an address; the master starts every transfer and drives the clock.

WIRES SDA (data), SCL (clock), + GND	**CLOCKING** Synchronous, master drives SCL	**DUPLEX · TOPOLOGY** Half · multi-drop bus	**ADDRESSING** 7-bit (10-bit optional)
SPEEDS 100 k · 400 k · 1 M · 3.4 Mbit/s	**BUS LIMIT** 400 pF (Sm/Fm), 550 pF (Fm+)	**REACH** On-board; short cables only	**ERROR CHECK** ACK/NACK per byte (no CRC)

Both lines are **open-drain**: a device can only pull a line LOW; pull-up resistors (R_p) return it HIGH. That is why any number of chips can share the two wires without fighting, and why a missing pull-up gives a dead bus.

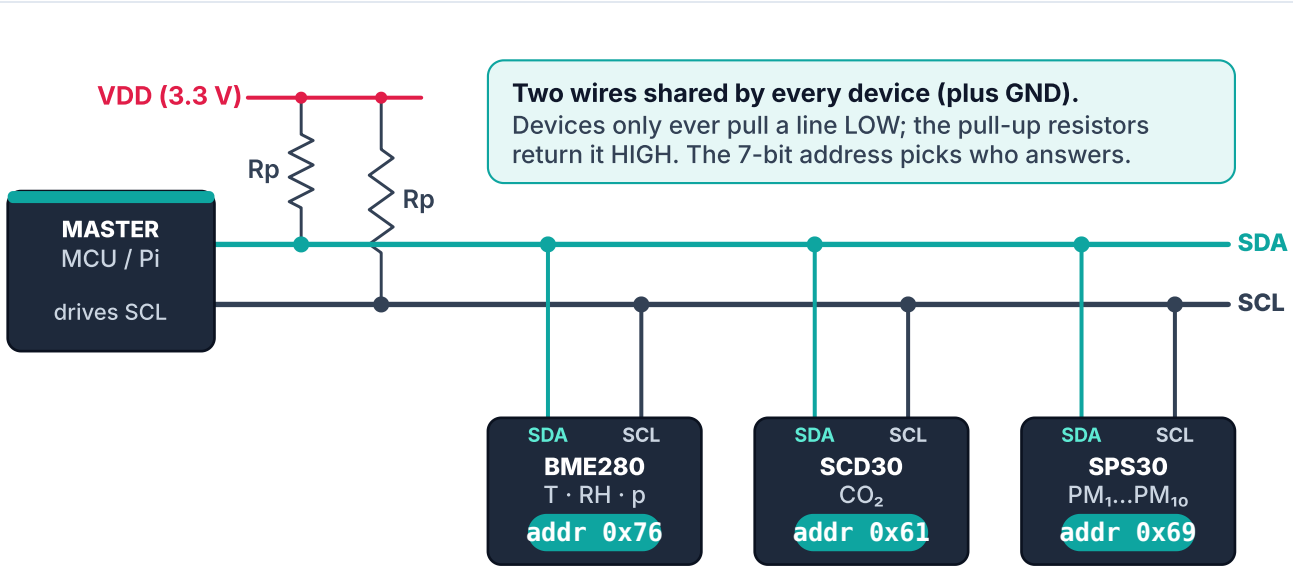
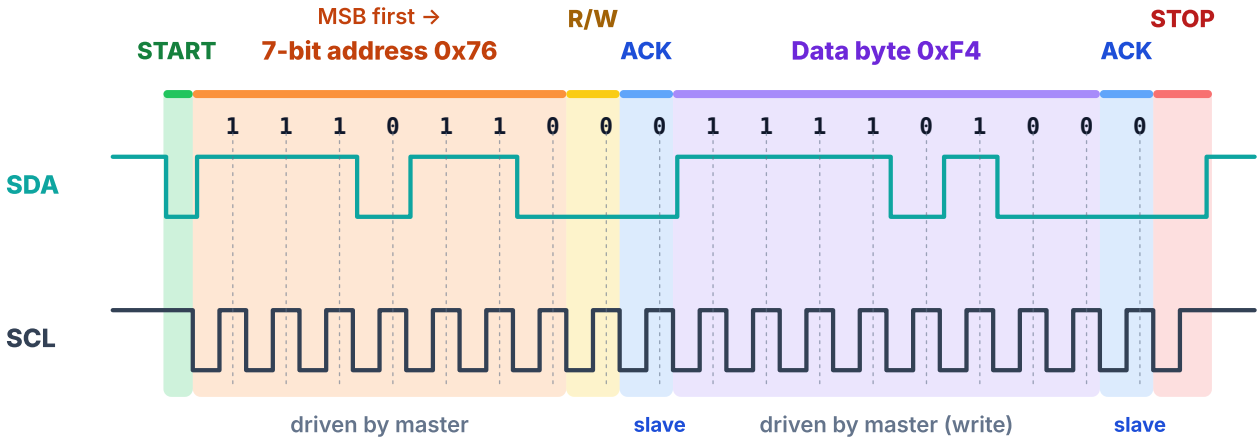


Figure 3. One master and three sensors on one I²C bus. The addresses are the datasheet defaults of the parts shown (BME280 0x76, SCD30 0x61, SPS30 0x69).

A transaction, step by step

1. **Idle.** SDA and SCL both HIGH.
2. **START (S).** The master pulls SDA LOW while SCL is HIGH. Every device starts listening.
3. **Address byte.** Seven address bits, MSB first, then one **R/W** bit (0 = master writes, 1 = master reads).
4. **ACK.** On the 9th clock the addressed device pulls SDA LOW. If nobody owns the address, SDA stays HIGH: a **NACK**.
5. **Data bytes.** 8 bits each, every byte acknowledged by the receiver. When reading, the master ACKs every byte except the last, which it NACKs to say "stop sending".
6. **STOP (P)** (SDA rises while SCL is HIGH) releases the bus. A **repeated START (Sr)** instead keeps the bus and lets the master change direction, e.g. write a register number then read it.



SDA may change only while SCL is LOW; it is read while SCL is HIGH (dotted lines).
START/STOP are the two exceptions: SDA falls (START) or rises (STOP) while SCL is HIGH.

Figure 4. Master writes to address 0x76. The first byte on the wire is 0x76 shifted left plus R/W = 0, i.e. 0xEC. Bit values are printed above SDA. Colour bands follow the convention of common teaching diagrams.

Reading one register: BME280 chip-ID (register 0xD0, value 0x60)

S	0xEC	A	0xD0	A	Sr	0xED	A	0x60	N	P
start	0x76 + W	slave	register	slave	rep. start	0x76 + R	slave	data (slave)	master NACK	stop

Most sensors work this way: write the register address, repeated START, read the value(s). Reading a known ID register is the first test of any new sensor.

Slave addressing

The 7-bit field gives 128 codes. Sixteen are reserved by the specification (0000 xxx and 1111 xxx: general call, START byte, CBUS, Hs-mode master codes, 10-bit addressing prefix 11110xx, device ID), leaving **112 usable addresses**. The address is fixed by the part number; many sensors offer one or two alternatives selected with a pin (BME280: SDO pin LOW → 0x76, HIGH → 0x77). Two identical sensors with no address pin cannot share a bus; Section 11 shows the fix (a multiplexer).

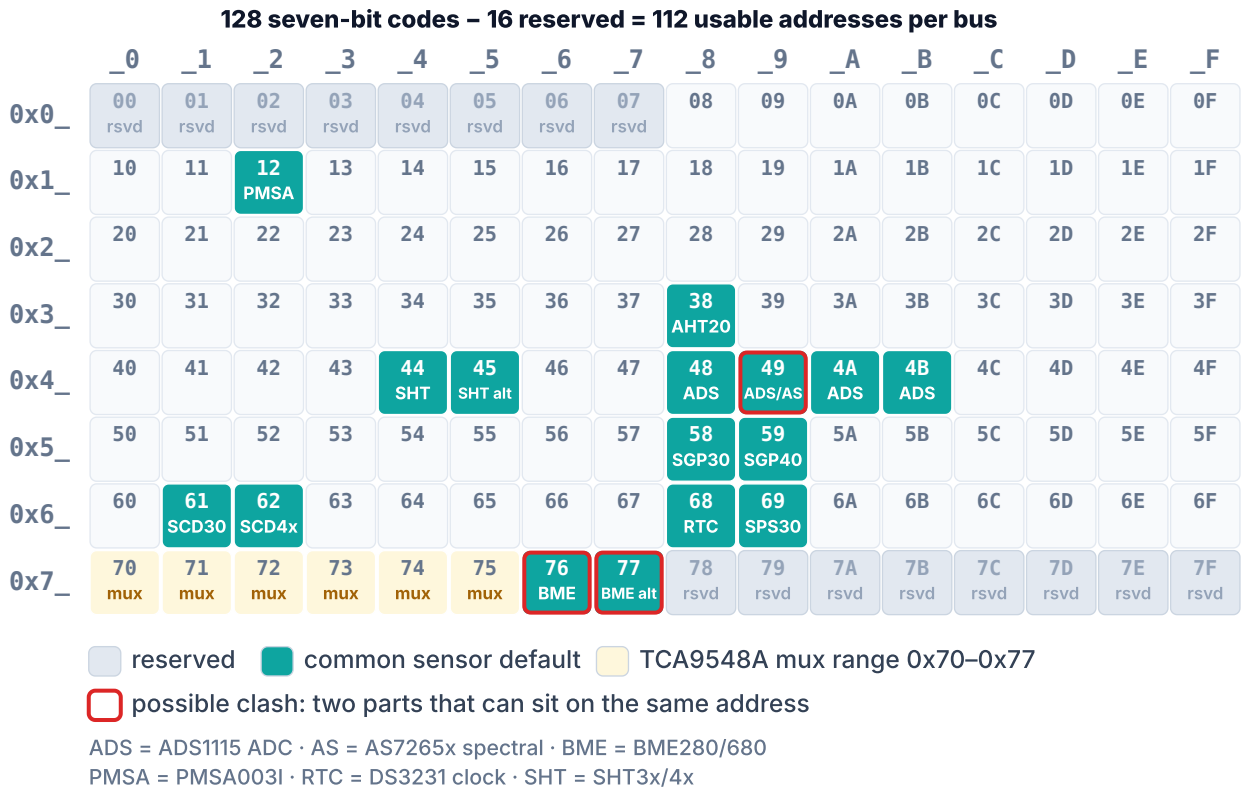


Figure 5. The 7-bit address space with common environmental parts at their default addresses. Red outlines mark real clashes: AS7265x and ADS1115 can both sit at 0x49, and a TCA9548A multiplexer (0x70–0x77) can collide with a BME280 at 0x76/0x77 unless set to a free address.

The "8-bit address" trap

Some datasheets and libraries quote the address already shifted with the R/W bit: 0x76 becomes 0xEC (write) and 0xED (read). Linux tools (`i2cdetect`, `smbus2`) and the Arduino `Wire` library expect the **7-bit** value 0x76. If a device NACKs at an address above 0x77, you are probably holding the 8-bit form. Also note: some popular diagrams draw the R/W bit before the address; on the wire it is always the 8th bit, after A6…A0.

What each mechanism achieves

Mechanism	What it achieves
START / STOP	Unambiguous frame boundaries: the only two moments SDA may change while SCL is HIGH, so they can never be confused with data.
7-bit address + R/W	Selects one device among many on two shared wires and states the direction of the transfer.
ACK / NACK	Confirms that someone received each byte; detects a missing or busy device. It does **not** detect corrupted bits (no checksum).
Clock stretching	A slow slave holds SCL LOW until it is ready. The SCD30 CO_2 sensor relies on it (12 ms typical, up to 150 ms rarely), so the master must support it.
Open-drain + pull-ups	Lets any device pull a shared line LOW without short circuits; enables ACK, stretching and multi-master arbitration.
Arbitration	If two masters start together, the one sending a 1 while the bus reads 0 detects the clash and backs off; the other's message survives intact.

Table 2. I^2C mechanisms (NXP UM10204, Rev. 7.0, 2021).

Mode	Bit rate	Max rise time t_r	Max bus capacitance	Typical use
Standard-mode (Sm)	≤ 100 kbit/s	1000 ns	400 pF	default; SCD30, SPS30 (100 kHz max)
Fast-mode (Fm)	≤ 400 kbit/s	300 ns	400 pF	most modern sensors
Fast-mode Plus (Fm+)	≤ 1 Mbit/s	120 ns	550 pF	needs 20 mA-capable drivers
High-speed (Hs)	≤ 3.4 Mbit/s	—	100 pF at 3.4 Mbit/s	rare in sensors
Ultra-Fast (UFm)	≤ 5 Mbit/s	—	—	write-only, push-pull

Table 3. I^2C speed modes. A bus runs at the speed of its slowest device.

SECTION 3

UART

Universal Asynchronous Receiver-Transmitter

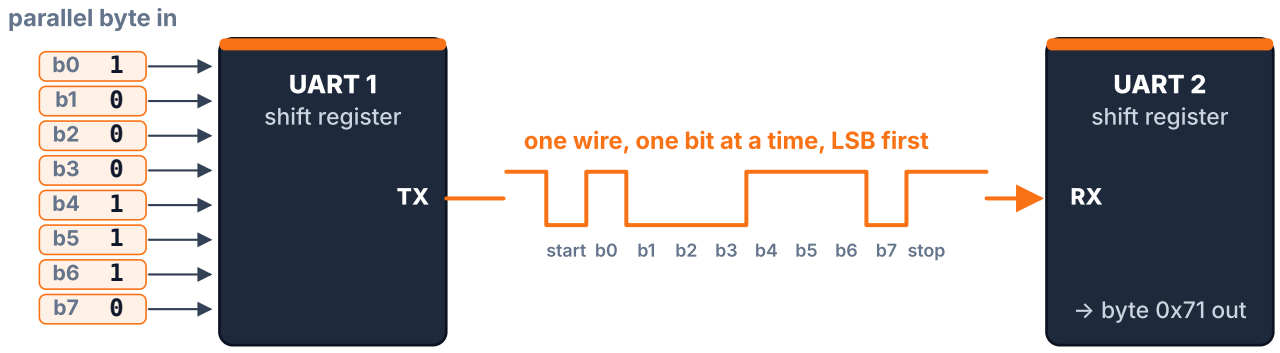
8N1

No clock wire. Both ends agree on a bit time in advance, and every character carries its own start and stop bits so the receiver can resynchronise.

WIRES	CLOCKING	DUPLEX · TOPOLOGY	ADDRESSING
TX, RX, GND (crossed)	Asynchronous, pre-agreed baud	Full · point-to-point	None (only two ends)

COMMON SPEEDS	FRAME	LEVELS	ERROR CHECK
9600 · 19200 · 115200 baud	start · 5–9 data · parity? · 1–2 stop	Logic (3.3 V / 5 V), idle HIGH	Optional parity, framing check

A UART is a shift register with a timer. The transmitter turns a parallel byte into a timed sequence of bits on one wire; the receiver times the bits back into a byte. Because there is no shared clock, the two ends must be configured identically: **baud rate, data bits, parity and stop bits**, written compactly as e.g. **9600 8N1** (9600 baud, 8 data bits, No parity, 1 stop bit) or **7E1** (7 data, Even parity, 1 stop).



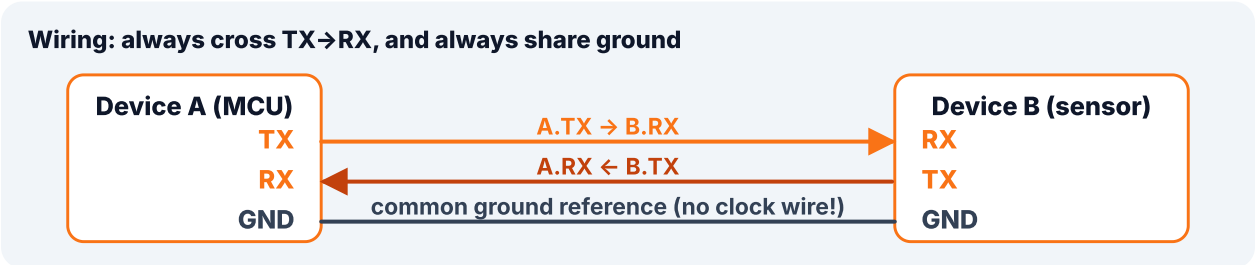


Figure 6. Parallel-to-serial conversion and the wiring rule: one device's TX goes to the other's RX, and both share ground. Connecting TX to TX is the most common first-day mistake.

Anatomy of a frame

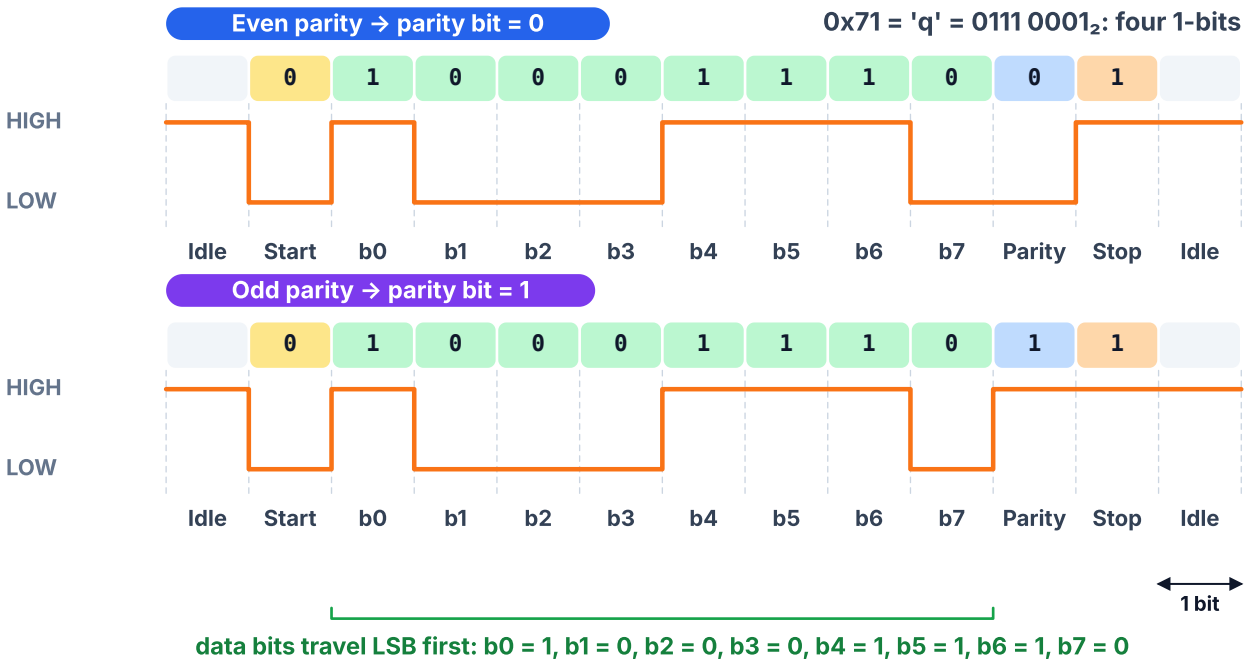


Figure 7. The character 'q' (0x71) sent with 8 data bits, a parity bit and one stop bit. With even parity the parity bit is 0 (the four 1-bits are already an even count); with odd parity it is 1.

Start bit (always 0)

The line idles HIGH, so the HIGH→LOW edge of the start bit is a guaranteed timing reference. The receiver restarts its bit timer on this edge for every character, so clock error can only accumulate over about 10 bits.

Stop bit(s) (always 1)

Return the line to idle so the next start edge is visible, and give the receiver a check: a 0 where a stop bit should be raises a **framing error** (usually wrong baud rate or noise).

Data bits (LSB first)

Normally 8. Seven-bit data (7E1) survives in ASCII instruments and SDI-12 sensors.

Parity bit (optional)

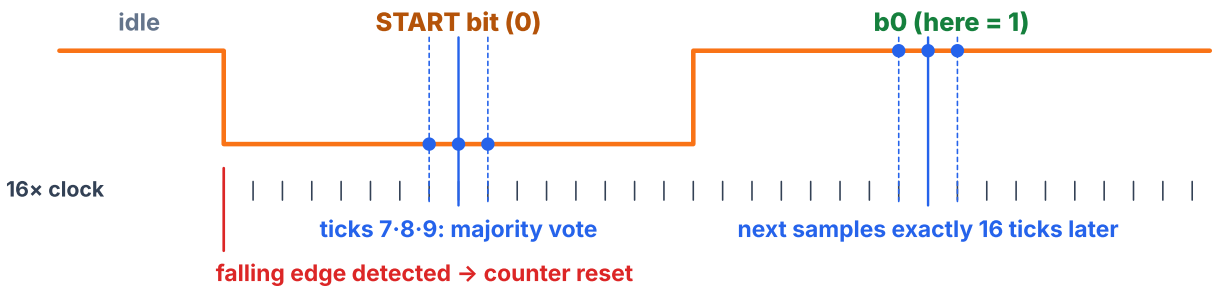
Even/odd parity makes the count of 1s even/odd. It catches any **odd** number of flipped bits in a character, misses an even number, and cannot correct anything.

Baud rate

The baud rate fixes the bit time. Since a 8N1 character uses 10 bits on the wire for 8 bits of data, the byte rate is baud ÷ 10.

Baud	Bit time	Bytes/s (8N1)	One 32-byte PMS5003 frame	Where you meet it
1200	833.3 µs	120	266.7 ms	SDI-12 sensors
9600	104.2 µs	960	33.3 ms	PMS5003, many GNSS defaults, Modbus
19200	52.1 µs	1 920	16.7 ms	Modbus RTU default, SCD30 Modbus
115200	8.68 µs	11 520	2.78 ms	SPS30 UART, debug consoles
921600	1.09 µs	92 160	0.35 ms	high-rate logging over USB bridges

Table 4. Standard baud rates, computed from bit time = 1/baud and 10 bits per 8N1 character.



The receiver's clock runs 16× faster than the baud rate and restarts on every START edge. Sampling each bit near its centre leaves about half a bit of slack for clock mismatch.

Figure 8. How the receiver finds bits without a clock: a 16× sampling clock, reset on the start edge, samples each bit near its centre (many UARTs take a majority vote of ticks 7, 8 and 9).

Error flag	Meaning	Usual cause
Framing error	Stop bit read as 0.	Baud mismatch, noise, or line inverted (RS-232 on a logic pin).
Parity error	Parity bit disagrees with the data.	Noise, or parity setting mismatch.
Overrun	A new byte arrived before software read the last one.	Reading too slowly; no FIFO/DMA; long interrupt latency.
Break	Line held LOW for longer than a whole frame.	Used deliberately as a wake-up signal (e.g. SDI-12), or a disconnected/shorted line.

Table 5. UART receive errors and what they usually mean.

UART is not RS-232

A microcontroller UART pin uses logic levels (0 V / 3.3 V or 5 V, idle HIGH). An RS-232 port uses inverted ±5…±15 V. Connecting an RS-232 port directly to a 3.3 V UART pin can destroy it, and even when it survives, the data reads as garbage because the logic is inverted.

SECTION 4

SPI

Serial Peripheral Interface

The fast, simple on-board bus: a clock, a data line in each direction, and a separate chip-select wire per device. No addresses, no acknowledgements.

SPI

WIRES	CLOCKING	DUPLEX · TOPOLOGY	ADDRESSING
SCLK, MOSI, MISO, GND + one CS per slave	**Synchronous, master drives SCLK**	**Full · one master, many slaves**	**Hardware chip select (active LOW)**

SPEEDS	MODES	REACH	ERROR CHECK
No fixed limit; ~1–10s of MHz (BME280 ≤ 10 MHz)	**4 (CPOL × CPHA)**	**On-board, short wires**	**None in the protocol**

SPI has no formal standard; it is a de facto convention that began at Motorola. Pin names vary: MOSI/MISO are also called SDI/SDO, DIN/DOUT, or COPI/CIPO; chip select (CS) is also SS, CE or NSS. The bar over a name (e.g. $\overline{\text{CS}}$) means active LOW.

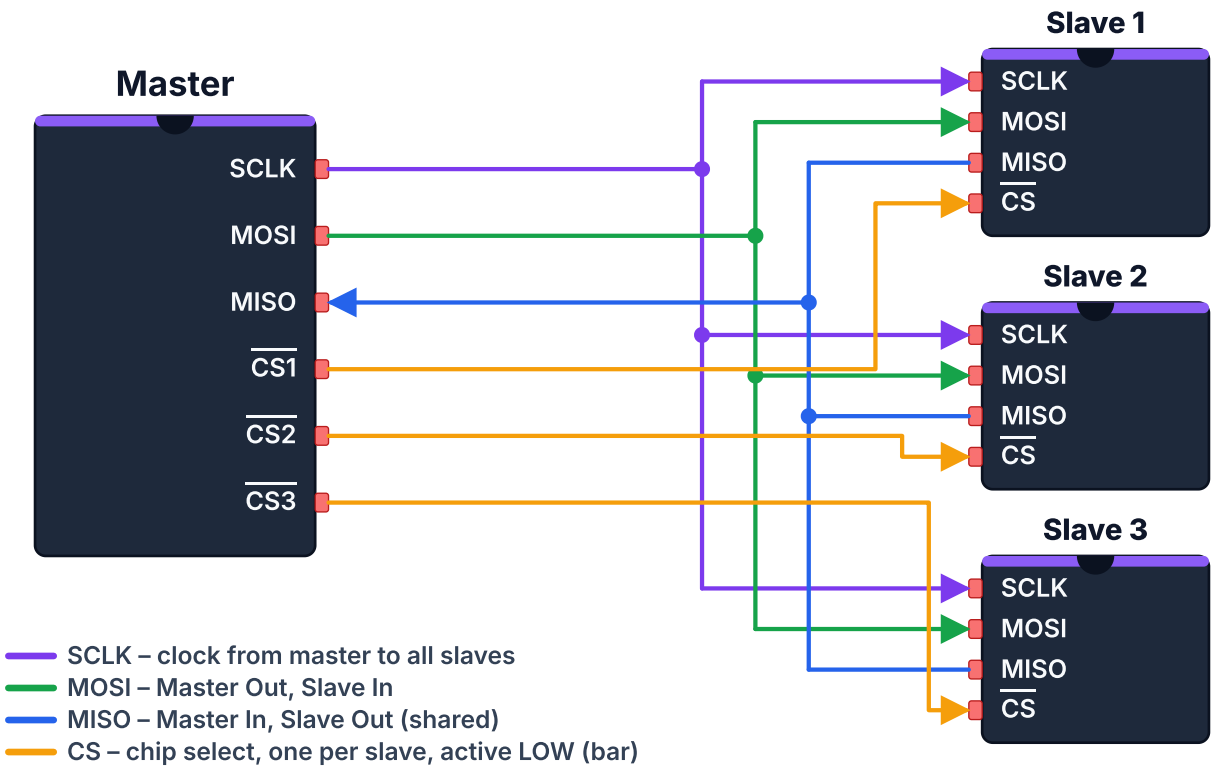


Figure 9. One master, three slaves. SCLK and MOSI fan out to all slaves; MISO is shared and each slave must release it (high impedance) when not selected; every slave gets its own chip-select line.

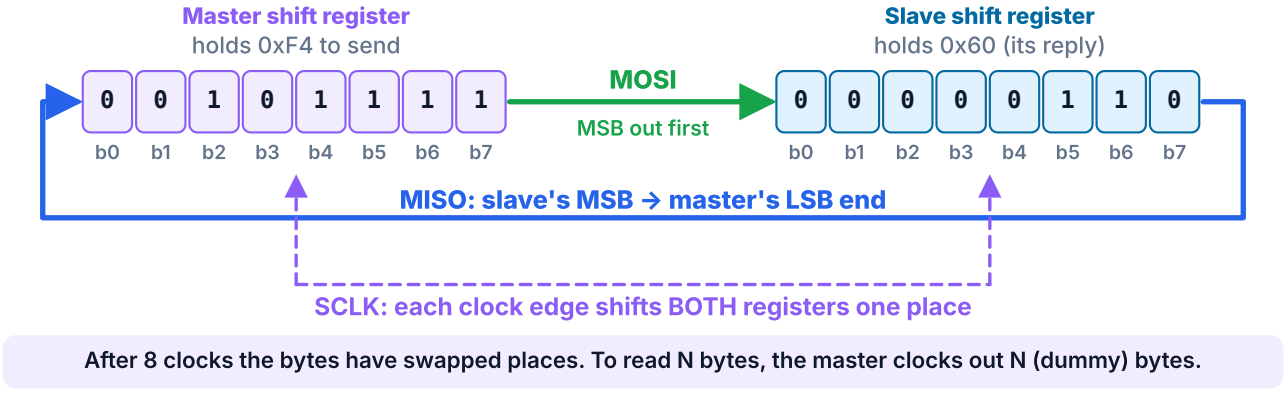


Figure 10. SPI is an exchange, not a request: master and slave shift registers form a ring, so every clock moves one bit each way. To read, the master must send something (often 0x00 or 0xFF "dummy" bytes).

The four modes: CPOL and CPHA

CPOL sets the clock's idle level; **CPHA** sets whether data is sampled on the first (leading) or second (trailing) clock edge after CS falls. The master must use the mode the slave's datasheet specifies.

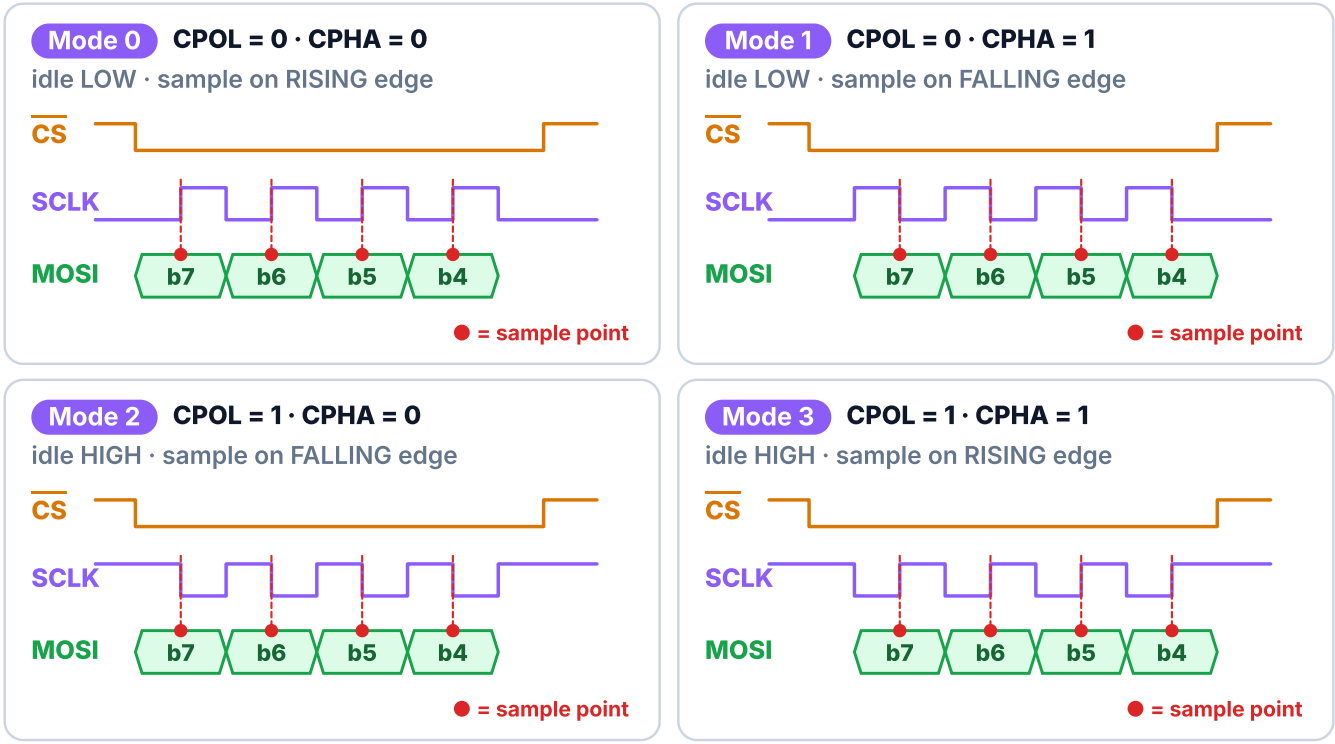


Figure 11. The four SPI modes. Red dots mark the edges on which the receiver samples; data changes on the opposite edges.

Mode	CPOL	CPHA	Clock idles	Data sampled on	Data shifted on	Examples
0	0	0	LOW	rising edge	falling edge	most common; BME280
1	0	1	LOW	falling edge	rising edge	Alphasense OPC-N3 (in open-source drivers)
2	1	0	HIGH	falling edge	rising edge	rare
3	1	1	HIGH	rising edge	falling edge	BME280 (supports 0 and 3)

Table 6. SPI modes (Analog Devices, "Introduction to SPI Interface").

What chip select achieves

It is the "address": only the slave whose CS is LOW drives MISO and listens to MOSI. CS falling also marks the start of a command and CS rising its end, so it frames transactions.

What SPI does not give you

No acknowledgement and no error check. With no slave connected the master still clocks in a byte, typically 0x00 or 0xFF. Verify presence by reading an ID register.

SECTION 5

USB

Universal Serial Bus & USB-to-serial bridges

A host-controlled, plug-and-play bus. For sensor work it usually appears as a "virtual COM port" that carries ordinary UART bytes.

WIRES (USB 2.0)	CLOCKING	DUPLEX · TOPOLOGY	ADDRESSING
VBUS 5 V, D+, D−, GND	**Embedded: NRZI + bit stuffing**	**Half · tiered star through hubs**	**7-bit, assigned by host (1–127)**

USB 2.0 SPEEDS	NEWER	REACH	ERROR CHECK
1.5 · 12 · 480 Mbit/s	**5–20 Gbit/s (USB 3.x), up to 80 Gbit/s (USB4 v2)**	**≈ 5 m per cable (FS/HS); hubs extend**	**CRC5/CRC16, ACK/NAK, retries**

Only the **host** (your PC or Raspberry Pi) starts transfers; devices answer when polled. Data travel in packets: a token from the host says who and what, a data packet carries the payload, a handshake (ACK/NAK/STALL) reports the outcome. Four transfer types exist: **control** (setup), **bulk** (large, error-checked, no timing guarantee; used by USB-serial), **interrupt** (small, polled at a guaranteed interval; keyboards) and **isochronous** (guaranteed bandwidth, no retries; audio).

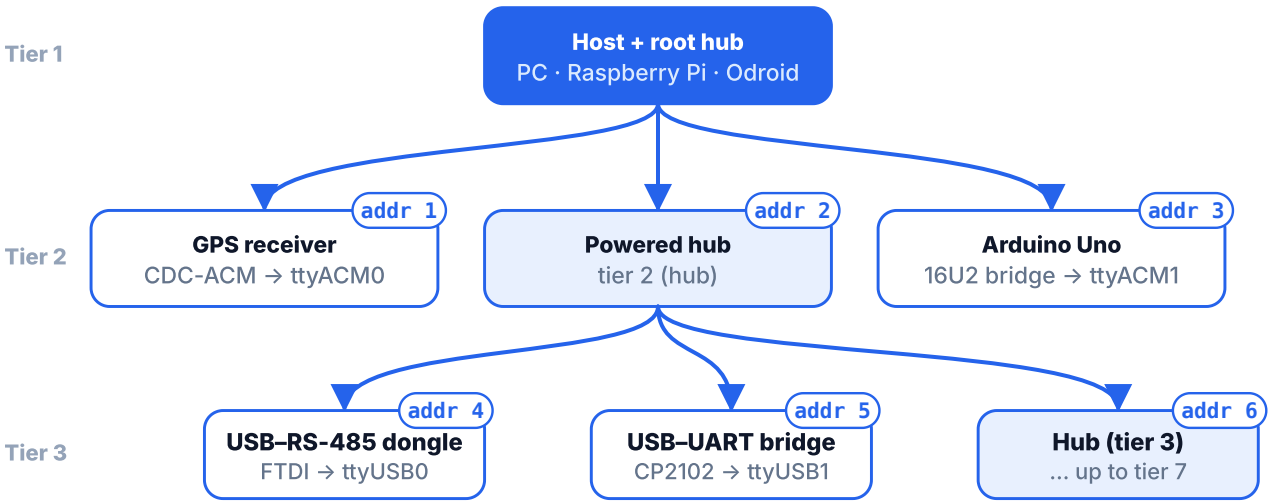


Figure 12. USB is a tree with the host at the root. Hubs add ports; each device gets a unique address when it is plugged in. Device names on the right are typical of a sensor node.

Enumeration: what happens when you plug a sensor in

1 Attach
A 1.5 kΩ pull-up on D+ (full/high speed) or D− (low speed) tells the host a device is present and its speed.

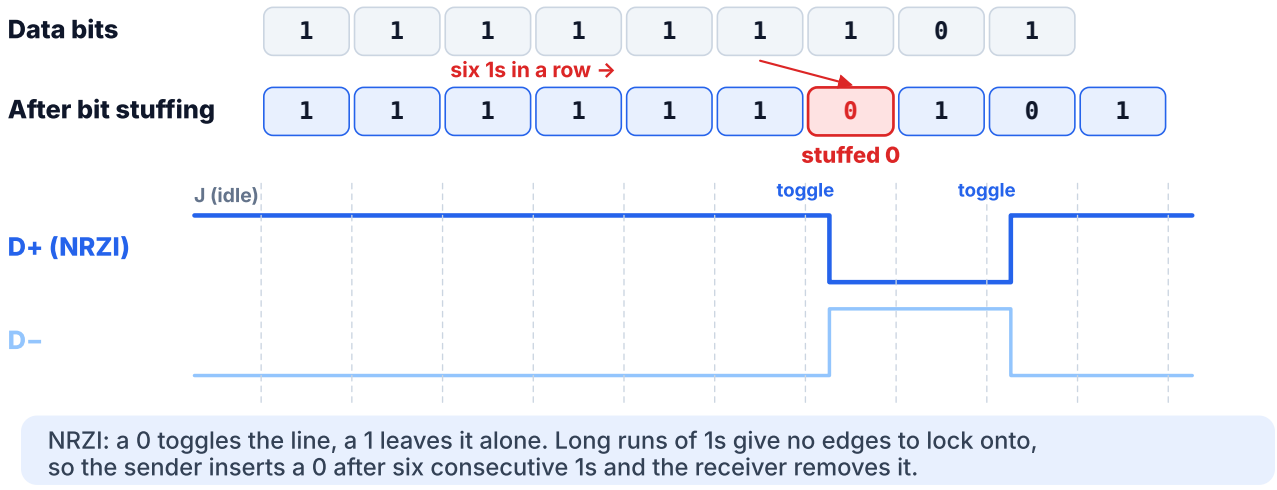
2 Reset
Host resets the port; the device answers at default address 0.

3 Describe
Host reads the device descriptor: vendor ID, product ID, class.

4 Address
Host assigns a unique address (1–127).

5 Configure
Host reads configuration, interface and endpoint descriptors.

6 Driver
OS loads a driver by VID:PID or class; a port such as /dev/ttyUSB0 appears.



NRZI: a 0 toggles the line, a 1 leaves it alone. Long runs of 1s give no edges to lock onto, so the sender inserts a 0 after six consecutive 1s and the receiver removes it.

Figure 13. USB carries its clock inside the data. NRZI encoding toggles the line for every 0; after six consecutive 1s the transmitter "stuffs" a 0 so the receiver always sees edges to stay locked.

Serial over USB

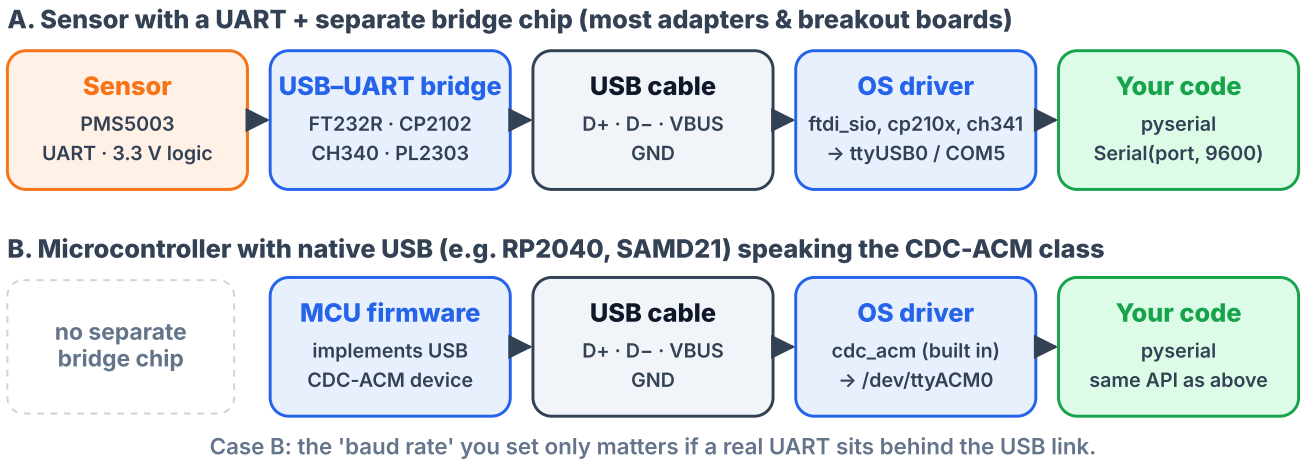


Figure 14. Two ways a sensor becomes a serial port on your computer. Either way your code just opens a port name and reads bytes.

Port names

Linux: vendor bridges (FTDI, Silicon Labs CP210x, WCH CH340) → `/dev/ttyUSB0`; CDC-ACM devices → `/dev/ttyACM0`. Windows: `COM3`, `COM5` … macOS: `/dev/cu.usbserial-…` / `/dev/cu.usbmodem…`.

Power on the cable

A USB 2.0 port supplies 5 V: 100 mA before configuration, up to 500 mA after; USB 3.x up to 900 mA; USB-C up to 1.5 A or 3 A at 5 V; USB Power Delivery negotiates more (up to 240 W). Fans and heaters in PM and gas sensors need this budget.

SECTION 6

RS-232

232

TIA-232-F serial interface

The classic "serial port": UART frames sent as large, inverted, bipolar voltages between exactly two devices. Still common on dataloggers and older meteorological instruments.

WIRES TXD, RXD, GND (+ up to 6 handshake)	**FRAME** UART frames, asynchronous	**DUPLEX · TOPOLOGY** Full · point-to-point	**LOGIC 1 (MARK)** −3 … −15 V at receiver
LOGIC 0 (SPACE) +3 … +15 V at receiver	**STANDARD RATE** ≤ 20 kbit/s (ICs often faster)	**CABLE** ≤ 2500 pF load (≈ 15 m)	**CONNECTOR** DE-9 (often called DB-9), DB-25

An RS-232 transceiver (e.g. MAX3232, which runs from 3.0–5.5 V and makes its ±voltages with a charge pump) sits between the UART pins and the cable. It inverts the signal and swings it well away from ground. The ±3 V dead band between the two valid ranges gives noise margin, but the signal is still single-ended, so ground shifts and cable capacitance limit it to short links.

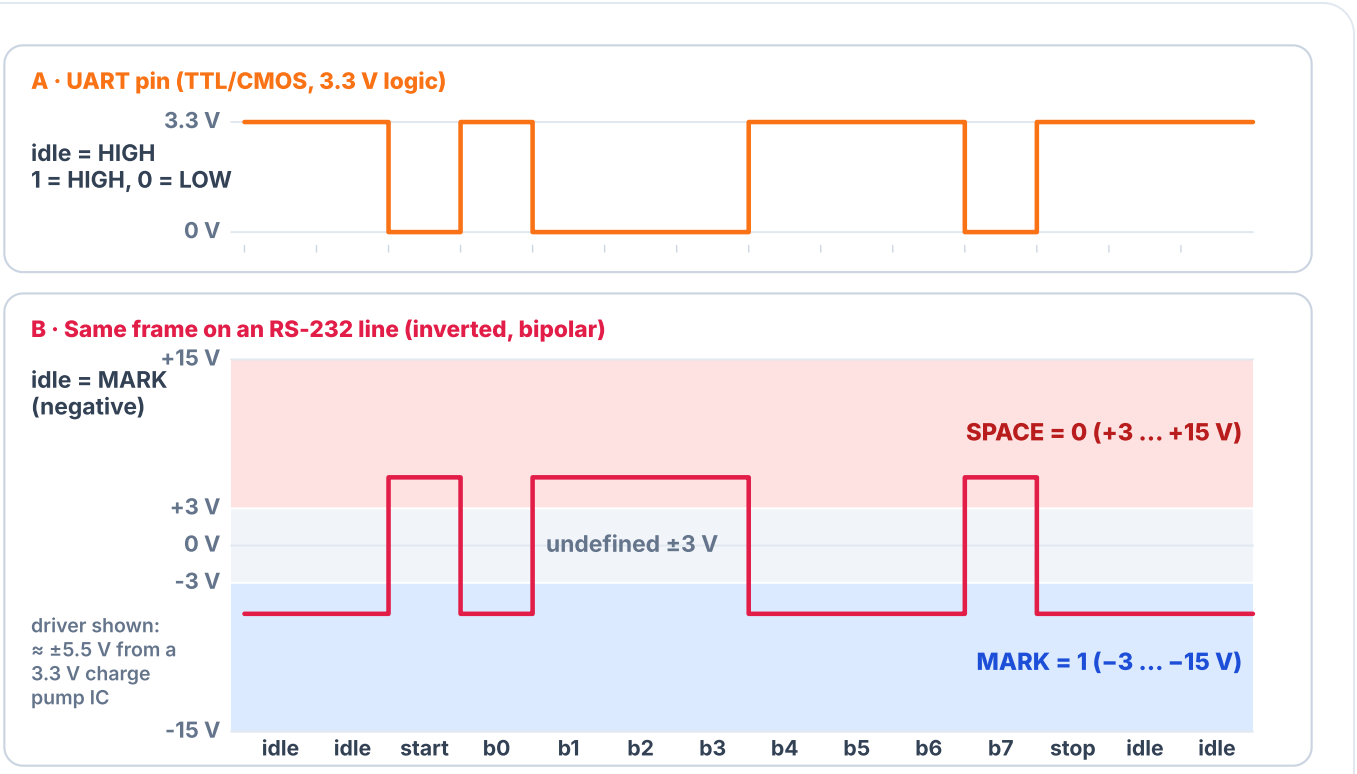


Figure 15. The same 8N1 frame for 0x71 at a UART pin (A) and on an RS-232 line (B). Logic 1 and idle are negative (MARK); logic 0 is positive (SPACE). Voltage bands are the TIA-232-F receiver thresholds.



DE-9 male, DTE (PC side), seen from the pin side
Female connectors are mirror-imaged: 5…1 on top

Pin	signal	direction (from the DTE)	
1	DCD	Data Carrier Detect	in
2	RXD	Receive Data	in
3	TXD	Transmit Data	out
4	DTR	Data Terminal Ready	out
5	GND	Signal ground	—
6	DSR	Data Set Ready	in
7	RTS	Request To Send	out
8	CTS	Clear To Send	in
9	RI	Ring Indicator	in

Minimum working link: TXD, RXD and GND (pins 2, 3, 5).

Figure 16. DE-9 pin-out for a DTE (computer-side) connector. Only pins 2, 3 and 5 are needed for most sensors; the rest are handshake and modem-status lines.

TIA-232-F parameter	Value	TIA-232-F parameter	Value
Driver output (loaded)	±5 … ±15 V	Max load capacitance	2500 pF
Receiver thresholds	±3 V (between: undefined)	Max slew rate	30 V/µs
Receiver input resistance	3 – 7 kΩ	Max data rate (standard)	20 kbit/s

Table 7. Key electrical limits of TIA-232-F (TI application report SLLA037A). Modern transceivers often exceed the 20 kbit/s figure on short cables.

DTE and DCE. RS-232 names pins from the viewpoint of the Data Terminal Equipment (the computer). The other end, Data Circuit-terminating Equipment (originally a modem), uses the same names but the opposite direction, so a straight cable connects DTE to DCE correctly. Two DTEs (e.g. a PC and a datalogger) need a **null-modem** cable that crosses the lines.

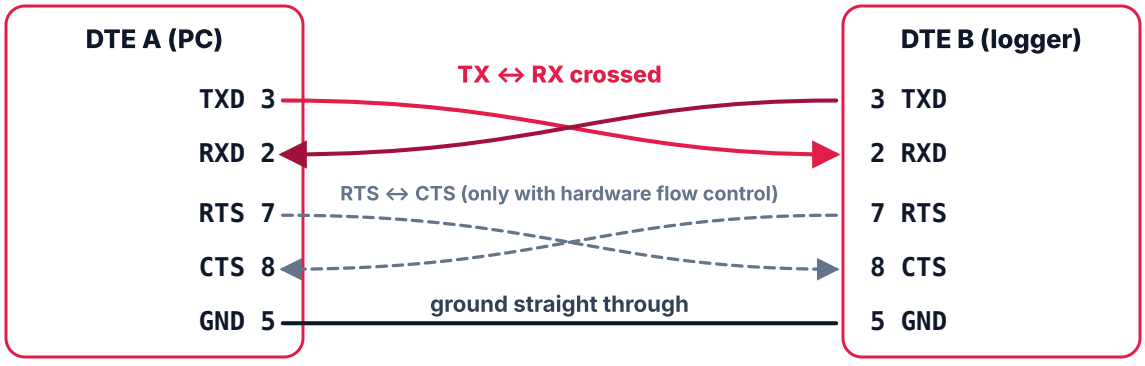


Figure 17. Null-modem wiring between two DTEs. Handshake lines matter only if the devices use hardware flow control.

SECTION 7

RS-485

485

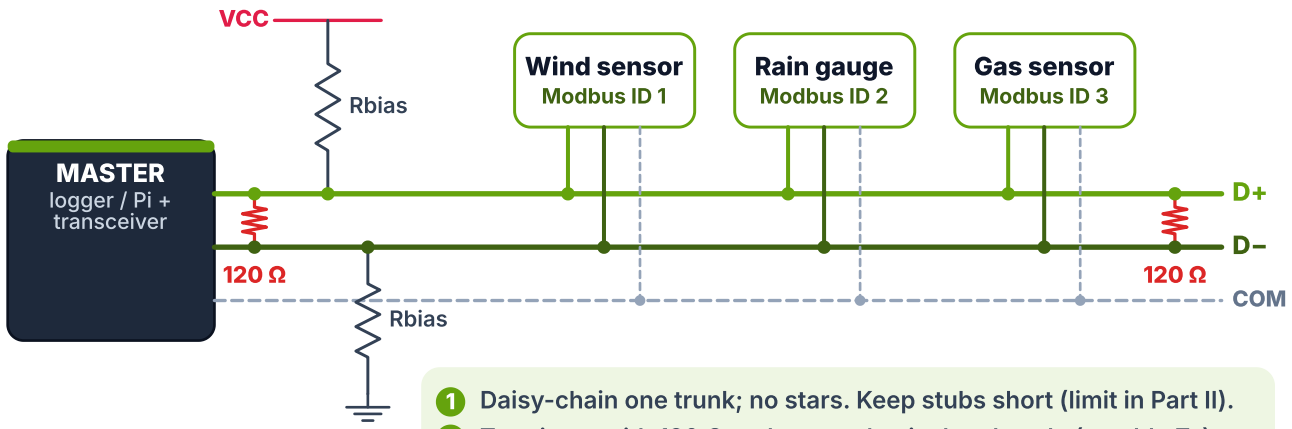
TIA-485-A balanced multipoint interface

A differential, multi-drop electrical standard: one twisted pair, up to 32 unit loads, up to 1200 m. The workhorse for outdoor meteorological and industrial sensors, usually running Modbus RTU.

WIRES	SIGNALLING	RECEIVER	COMMON-MODE RANGE
Twisted pair A/B + common (2-wire); 4-wire = full duplex	Differential: driver ≥ 1.5 V into 54 Ω	Threshold ±200 mV	−7 V … +12 V

NODES	SPEED · REACH	ADDRESSING	ERROR CHECK
32 unit loads (≤ 256 with ⅛-UL parts)	10 Mbit/s (≈ 12 m) … ≈ 100 kbit/s at 1200 m	None electrically; protocol adds IDs	None electrically; Modbus adds CRC-16

The receiver looks only at the **difference** between the two wires. Interference couples almost equally into both wires of a twisted pair, so it cancels; a ground difference between distant buildings also cancels, as long as each line stays within −7 V … +12 V of the receiver's ground.



1. Daisy-chain one trunk; no stars. Keep stubs short (limit in Part II).
2. Terminate with 120 Ω at the two physical ends only (≈ cable Z_0).
3. Bias the idle bus once (one pull-up/pull-down pair), so receivers see a defined level when no driver is talking.
4. Carry a common/reference wire, or isolate nodes (see Part II).

Figure 18. A two-wire RS-485 bus: one trunk daisy-chained past every node, 120 Ω terminations at both physical ends, one bias pair, and a common reference conductor. Only one node may drive at a time.

The A/B labelling trap
TIA-485 says that for logic 1 (mark, idle) terminal A is negative relative to B. Many transceiver makers print the labels the other way round (A = non-inverting). This guide draws the lines as **D+** (high for idle) and **D–**. If a correctly configured link is silent, swap the pair; it cannot damage anything.

COMPUTED WAVEFORM

Why differential signalling survives noisy cables

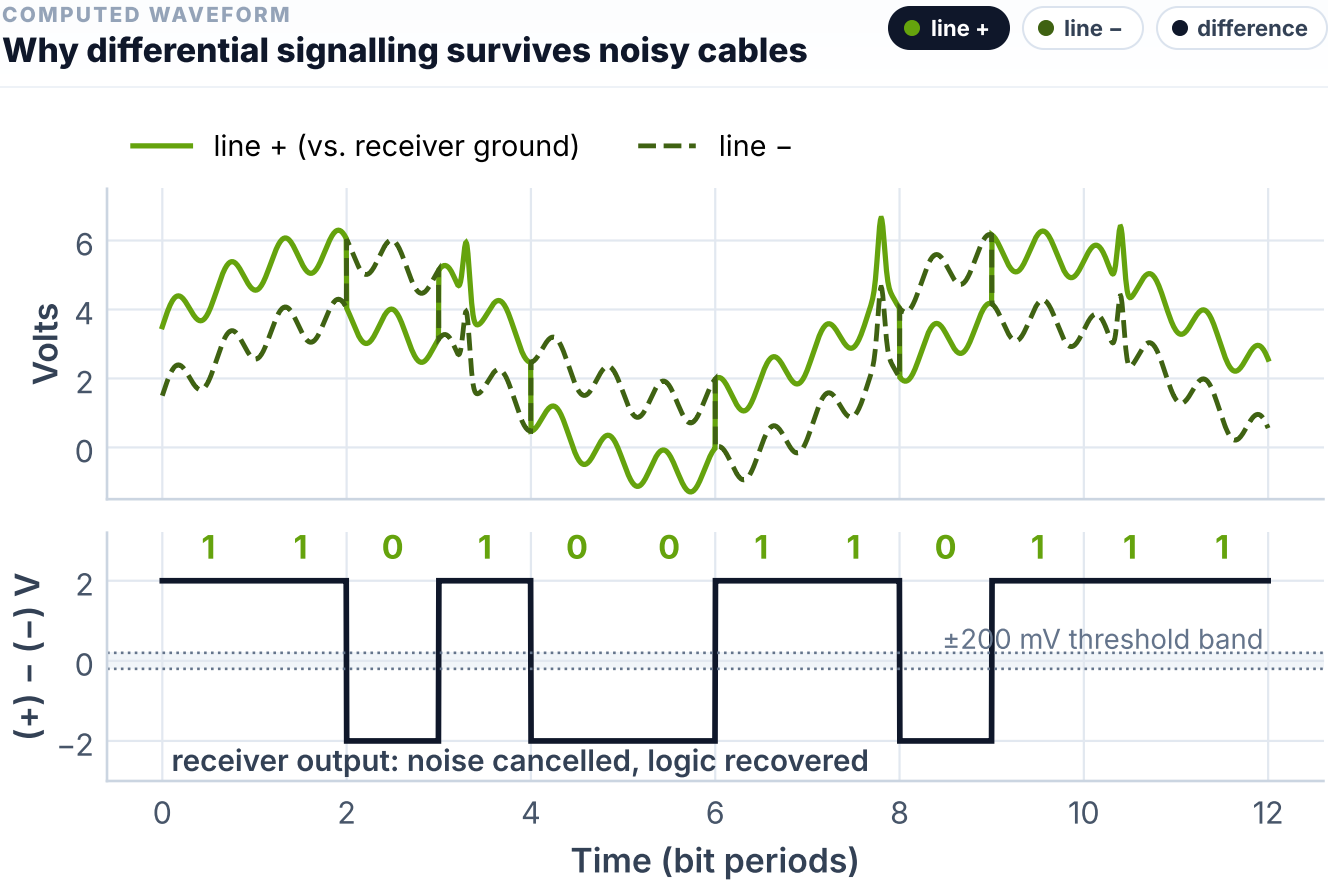


Figure 19. Both lines carry several volts of shared noise (slow ground shift, hum and spikes). Their difference is a clean ±2 V signal, far above the ±200 mV threshold. **Simulated**

Half-duplex turn-taking

Each transceiver has a driver-enable (DE) and receiver-enable ($\overline{\text{RE}}$) pin. The master enables its driver, sends a request, releases the line; the addressed slave then drives its reply. Only one driver may be on at a time.

RS-422 vs RS-485

RS-422 (TIA-422) allows one driver and up to 10 receivers: fine for a sensor that streams to one logger. RS-485 allows many drivers on the same pair, which is what a multi-sensor bus needs.

Addressing comes from the protocol: Modbus RTU in one minute

RS-485 moves bits; it has no addresses. **Modbus RTU** adds them: the master sends a frame starting with a slave ID (1–247; 0 = broadcast), a function code and data, closed by a CRC-16; only the addressed slave replies. Frames are separated by at least 3.5 character times of silence.

Request: "slave 1, read 2 holding registers starting at 0"

01	03	00 00	00 02	C4 0B
slave ID	function	start reg	count	CRC (lo, hi)

Reply: two registers, 0x00FA = 250 and 0x012C = 300

01	03	04	00 FA	01 2C	DA 4F
slave ID	function	byte count	reg 0	reg 1	CRC (lo, hi)

CRCs computed with the Modbus CRC-16 (reflected polynomial 0xA001, initial value 0xFFFF). If the device's manual says the registers are scaled ×10, these mean 25.0 and 30.0. Section 15 covers timing and wiring.

SECTION 8

Side by side: similarities & differences

The same few ideas (clocking, framing, selection, error checking, electrical layer) solved six different ways.

	I²C	UART	SPI	USB 2.0	RS-232	RS-485
Signal wires	SDA, SCL, GND	TX, RX, GND	SCLK, MOSI, MISO, GND + 1 CS per slave	VBUS, D+, D−, GND	TXD, RXD, GND (+ handshake)	A, B + common; 4-wire for full duplex
Clock	sync (SCL)	async, agreed baud	sync (SCLK)	embedded (NRZI + stuffing)	async	async
Duplex	half	full	full	half	full	half (2-wire) / full (4-wire)
Topology	multi-drop bus	point-to-point	1 master, CS per slave	tiered star (hubs)	point-to-point	multi-drop daisy chain
Who may start	master	either end, any time	master	host only	either end	master polls (by protocol)
Device selection	7-bit address in every transfer	none needed	chip-select line	address assigned at enumeration	none needed	protocol ID (Modbus 1–247)
Electrical	open-drain, pull-ups, logic levels	push-pull logic levels	push-pull logic levels	differential	single-ended, inverted ±5…15 V	differential, ±200 mV threshold
Typical rate	100–400 kbit/s	9.6–115.2 kbaud	1–10+ MHz	12 / 480 Mbit/s	≤ 20 kbit/s (std); 115.2 kbaud common	9.6–115.2 kbaud; ≤ 10 Mbit/s
Practical reach	on-board, short cables	~1–2 m at logic levels	on-board, tens of cm	≈ 5 m per cable	≈ 15 m (2500 pF)	up to 1200 m

	I²C	UART	SPI	USB 2.0	RS-232	RS-485
Devices per bus	112 addresses (capacitance-limited)	2	limited by CS pins	127	2	32 UL (≤ 256)
Built-in error detection	ACK/NACK only	optional parity; framing	none	CRC5/CRC16, handshake, retry	as UART	none (protocol adds CRC)
Defined by	NXP UM10204	de facto	de facto (Motorola origin)	USB-IF specs	TIA-232-F (1997)	TIA-485-A (1998)
Sensor examples	BME280, SCD30, SPS30, SHT4x	PMS5003, SPS30, GNSS	OPC-N3, BME280, ADCs	GNSS, Arduino, USB bridges	dataloggers, legacy met instruments	Modbus wind, rain, soil sensors

Table 8. Master comparison. "Practical reach" for I²C, UART and SPI reflects typical engineering practice; the specifications themselves limit capacitance or timing, not metres.

What they have in common

Shared idea	Protocols	Why it matters
Bits travel one at a time on each data line	all six	Fewer wires and pins than parallel buses; timing becomes the central problem.
Identical character frame	UART, RS-232, RS-485	Start · data · parity · stop is the same everywhere; only the electrical layer differs, so a transceiver converts between them with no software change.
Master-driven clock, register-map devices	I²C, SPI	Both are on-board buses where the master reads/writes numbered registers in the sensor.
Many devices share wires; one is selected	I²C, SPI, USB, RS-485 (+ Modbus)	Selection by address (I²C, USB, Modbus) or by a dedicated wire (SPI CS).
Only the master/host starts a conversation	I²C, SPI, USB, Modbus on RS-485	Polling makes timing predictable; slaves never collide. Plain UART sensors may instead stream unprompted (PMS5003 active mode).
Differential signalling	RS-485, USB	Noise and ground offsets cancel, enabling longer cables and higher speeds.
A shared reference	all (within limits)	Single-ended links need a common ground; differential links still need common-mode within range (−7…+12 V for RS-485).

Table 9. Similarities across the six links.

The building blocks and what each one achieves

Mechanism	Found in	What it achieves	Without it…
Slave address	I²C, USB, Modbus	Lets many devices share the same wires; only the addressed one answers.	One wire set per device, or collisions.
Chip select	SPI	Hardware selection with zero protocol overhead; also frames each transaction.	Slaves fight on MISO.
Baud rate	UART, RS-232, RS-485	Fixes the bit time so a receiver without a clock wire knows when to sample.	Garbage or framing errors.
Start bit	UART family	Guaranteed edge that resynchronises the receiver at every character.	Clock error would accumulate without bound.
Stop bit(s)	UART family	Returns the line to idle so the next start edge is visible; enables framing-error detection.	Back-to-back bytes could not be separated.
Parity bit	UART family	Detects any odd number of bit errors in a character, at a cost of one bit.	Single-bit errors pass silently.
ACK / NACK	I²C, USB	Confirms delivery byte-by-byte (I²C) or packet-by-packet (USB); detects absent devices.	Master cannot tell if anyone heard.
Clock line	I²C, SPI	Receiver samples exactly when told; speed can vary or pause.	Need an agreed baud rate instead.
CRC / checksum	USB, Modbus, Sensirion I²C, PMS5003	Detects corrupted messages far more reliably than parity.	Bad readings enter your dataset.
Pull-up resistors	I²C	Return open-drain lines HIGH; set rise time.	Bus stuck LOW or too slow.
Termination (120 Ω)	RS-485	Absorbs the wave at cable ends so it does not reflect back.	Ringing and corrupted edges on long or fast lines.
Fail-safe bias	RS-485	Holds the idle bus at a valid level when no driver is on.	Receivers output random bits between frames.

Table 10. The mechanisms named in this guide, and what each one buys you.

Choosing a link

Sensor on the same board or within ~0.3 m
I²C if the part offers it and speeds are modest; it saves pins and lets you add sensors later.

High data rate, on-board
SPI for ADCs, displays, flash memory, particle counters with SPI. Budget one CS pin per device.

One sensor that streams, < ~1–2 m
UART at logic level. Prefer it over I²C for off-board cables (Sensirion recommends UART beyond 20 cm for the SPS30).

Sensor or MCU to a computer
USB with a USB-serial bridge or CDC-ACM device; also powers the sensor.

Outdoor, long cables, several sensors
RS-485 + Modbus RTU: one twisted pair, hundreds of metres, noise-tolerant, addressable.

Instrument with a DE-9 port
RS-232 as provided; add a USB-RS232 adapter (true ±V levels) on the logger.

CHART · APPROXIMATE ENVELOPES

Bit rate versus cable length

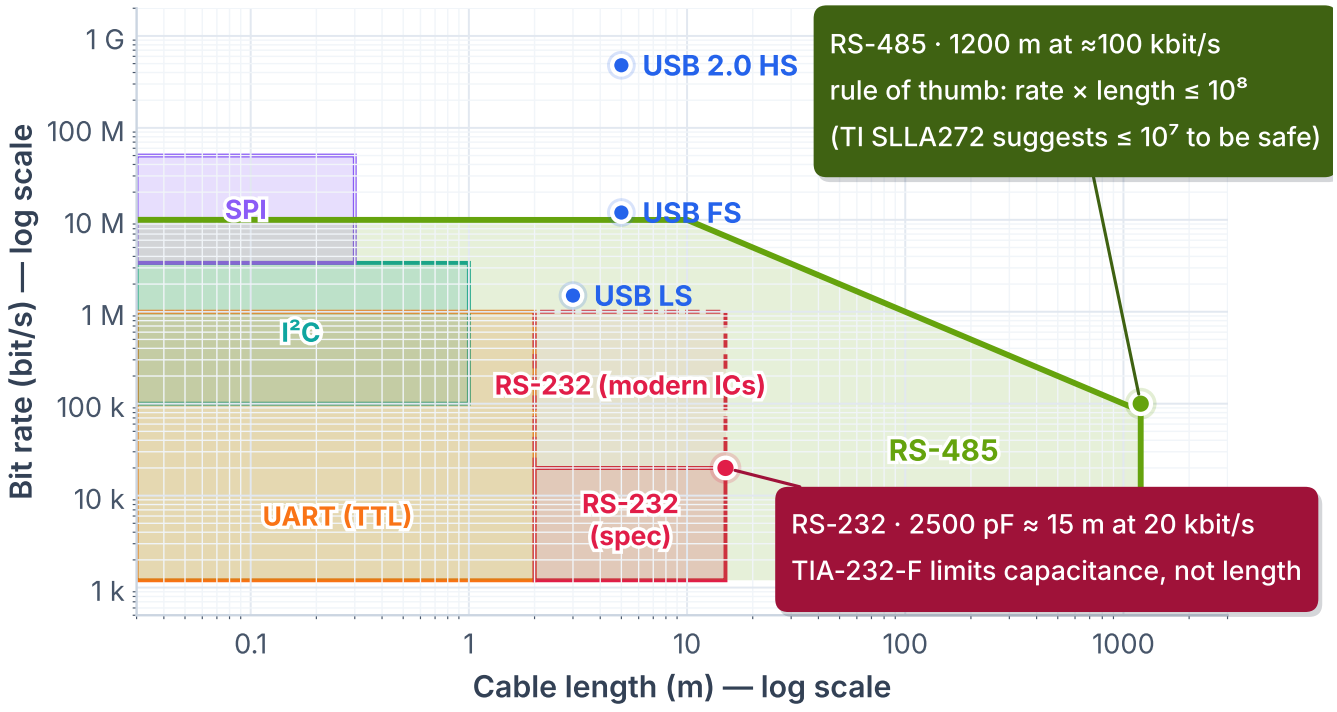


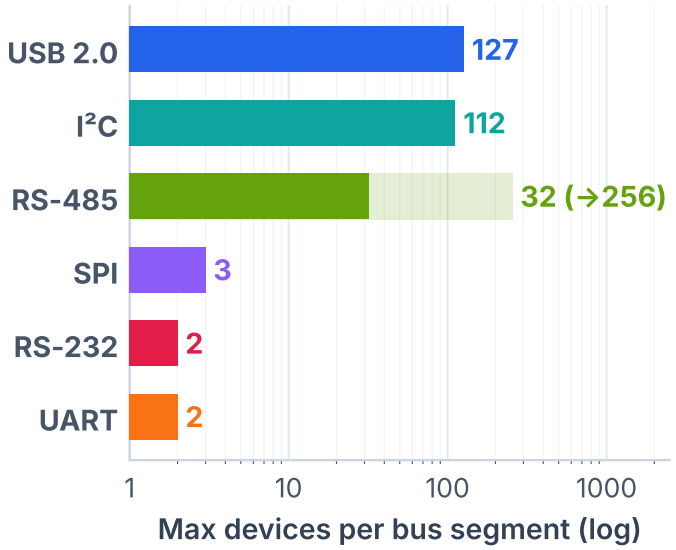
Figure 20. Where each link is comfortable. RS-485 follows the common rate × length ≤ 10^8 guideline with a 10 Mbit/s cap; RS-232 is bounded by 2500 pF; USB points are 2.0 speeds at their cable limits. On-board buses are shown at typical practice.

TIA-232-F · TIA-485-A · TI SLLA272

CHART

How many devices can share one bus?

log scale



127 addresses per host controller (incl. hubs)

112 usable addresses; 400 pF usually limits first

32 unit loads; 256 with ⅛-UL transceivers

one CS pin per slave (3 shown)

point-to-point: one driver, one receiver

point-to-point

Figure 21. Upper limits per bus segment. In practice I²C runs out of capacitance budget, and SPI runs out of chip-select pins, long before these numbers.

UM10204 · TIA-485-A · USB 2.0

PART II

Practice & advanced topics

02

What separates a bench demo from a node that logs clean data for a year: correct electrical design, the arithmetic behind pull-ups, baud rates and bus biasing, robust parsing, and a disciplined way to debug.

9–10 · System & electrical
sensor-node architecture, logic levels, level shifting, back-powering, grounding, isolation, protection

11–13 · On-board buses
pull-up sizing, capacitance budgets, clock stretching on Raspberry Pi, baud error, frame parsing, SPI timing

14–15 · Cables
USB-serial naming and latency, RS-232 practice, RS-485 termination, biasing, stubs, direction control, Modbus RTU timing

16–17 · Data integrity & formats
parity vs checksums vs CRCs, NMEA 0183 and SDI-12 on meteorological instruments

18–19 · Debug & deploy
tools, a troubleshooting flow, symptom table, field checklist and formula sheet

How to use it
Each section is self-contained; numbers are computed, and sources are listed at the end.

SECTION 9

Anatomy of a sensor node

A typical air-quality / meteorological node mixes every link in this guide. Knowing which part speaks which interface, and with which defaults, is the first design step.

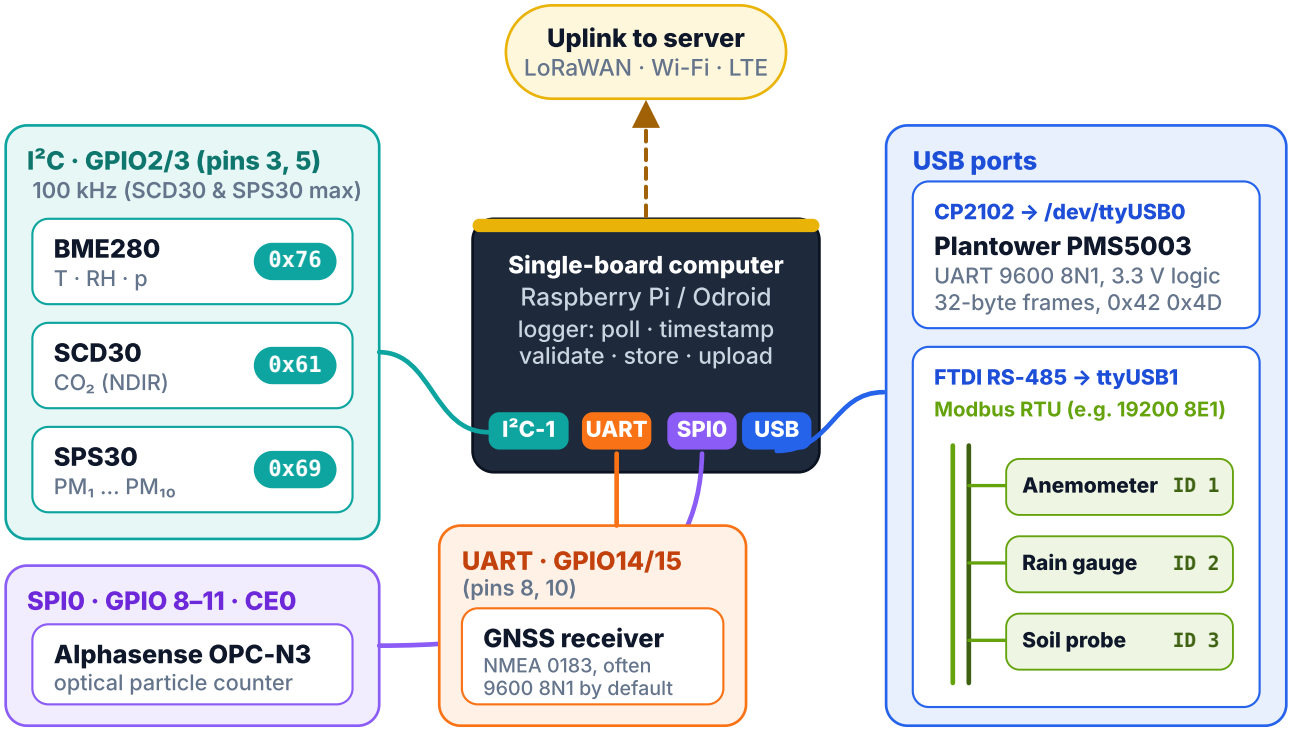


Figure 22. An illustrative node built around a single-board computer. Pin numbers are the Raspberry Pi 40-pin header (I²C-1 on GPIO2/3, UART on GPIO14/15, SPI0 on GPIO8–11). Each sensor is connected through its datasheet-default interface.

Part	Measures	Interface(s) and defaults	Watch out for
Bosch BME280	T, RH, pressure	I²C 0x76/0x77 (SDO pin), up to 3.4 MHz; SPI modes 0 and 3, up to 10 MHz	Chip-ID register 0xD0 reads 0x60; self-heating if sampled too often
Sensirion SCD30	CO_2 (NDIR), T, RH	I²C 0x61, ≤ 100 kHz with clock stretching; Modbus RTU 19200 8N1 (SEL pin to VDD)	Clock stretching (12 ms typical, up to 150 ms); CRC-8 on every word
Sensirion SPS30	PM_1...PM_{10}	UART (SHDLC) 115200 8N1 with SEL floating; I²C 0x69, ≤ 100 kbit/s with SEL to GND	Vendor recommends UART for cables longer than 20 cm

Part	Measures	Interface(s) and defaults	Watch out for
Plantower PMS5003	PM_1, $PM_{2.5}$, PM_{10}, counts	UART 9600 8N1, 3.3 V logic (5 V supply); active (streaming) or passive (on request) mode	32-byte frame, header 0x42 0x4D, 16-bit sum checksum
Alphasense OPC-N3	particle size spectrum	SPI (mode 1 in open-source drivers, e.g. 500 kHz)	Vendor command protocol needs delays between bytes
GNSS receiver	position, time	UART NMEA 0183 (often 9600 8N1 by default); many also enumerate as USB CDC-ACM	Sentences arrive continuously; parse by line and checksum
Professional met sensors	wind, rain, T/RH	Commonly RS-485 (Modbus RTU), RS-232 or SDI-12; model-specific	Read the manual for IDs, register maps and scaling

Table 11. Interfaces of representative parts, from their datasheets/interface descriptions. Defaults can be changed on many parts; always confirm on the unit you have.

Design rule of thumb
Put each long-cable or noisy sensor on its own link (UART, USB, RS-485) and keep I^2C/SPI for parts on the same board or a short ribbon. One misbehaving I^2C device can hold the whole bus LOW; a USB or RS-485 sensor fails alone.

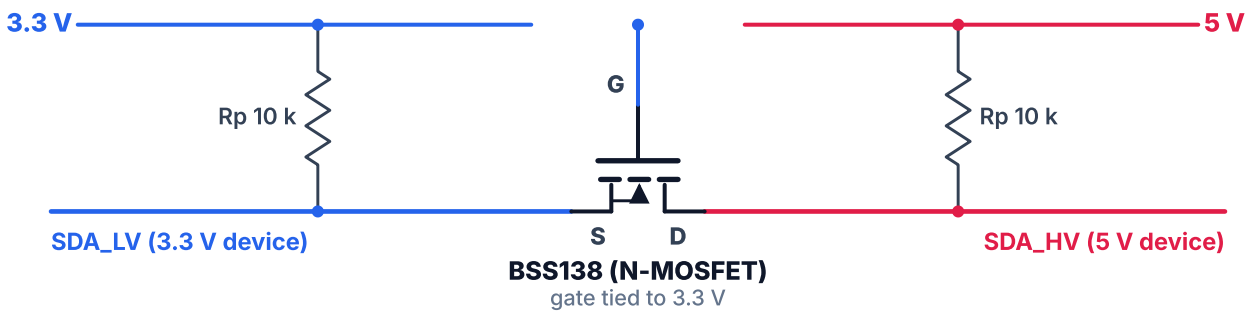
SECTION 10

Electrical ground rules

Most "protocol" problems in the field are electrical: wrong voltage levels, missing grounds, back-powered chips, noise and surges.

Logic levels and level shifting

Raspberry Pi GPIO pins are 3.3 V and **not 5 V tolerant**. Many sensors need a 5 V supply for a fan, laser or heater but use 3.3 V logic (PMS5003); others accept either (SPS30). Check two numbers in every datasheet: the I/O voltage and the absolute-maximum input voltage.



Bidirectional for open-drain lines (I²C, SMBus): either side pulls LOW → the other follows.
Circuit from Philips AN97055. For fast push-pull SPI use a direction-specific buffer instead.

Figure 23. The classic bidirectional level shifter for open-drain buses (one MOSFET and two pull-ups per line). Many I²C breakout boards contain exactly this circuit.

Signal type	Good choices	Avoid
I²C / open-drain, bidirectional	MOSFET circuit above; PCA9306; TXS0102 (auto-direction, open-drain capable)	TXB-type auto-direction translators (not intended for open-drain or pulled-up lines)
UART, SPI (push-pull, one direction per wire)	dual-supply buffers such as SN74LVC1T45/2T45; 74AHCT125 for 3.3 V → 5 V	Driving a 3.3 V-only input from a 5 V output
Slow 5 V → 3.3 V input only	resistor divider (e.g. 1 kΩ / 2 kΩ) on a low-baud RX line	Dividers on fast SPI clocks (they slow the edges)
RS-232 ↔ logic	MAX3232-class transceiver (3.0–5.5 V, 0.1 µF pumps)	Any direct connection

Table 12. Level-translation options by signal type.

Back-powering ("phantom powering")
If a sensor is unpowered but its I/O pins are connected to a live bus, current flows through its input protection diodes and partially powers it. The chip can latch up, hold the bus LOW, or never reset properly. Power all devices on a bus from the same switched rail, or use buffers/isolators that block current when one side is off.

Grounding, noise and isolation

One reference, short returns

Every single-ended signal returns through ground. Run a ground wire with every signal cable (ideally twisted with it), and keep high-current returns (fans, heaters, pumps, radios) away from sensor grounds.

Ground loops

Two devices earthed at different points (e.g. a mast-mounted sensor and an indoor PC) can differ by volts. Single-ended links (UART, I²C, RS-232) then carry that difference as noise. Fix with differential links, or with galvanic isolation (digital isolators, isolated RS-485 transceivers, USB isolators).

Protection at the connector

Outdoor cables collect ESD and induced surges. Put TVS diodes at the board edge, sized for the interface (RS-485 parts such as the asymmetric SM712 match the −7 V/+12 V range), and keep the protection-to-ground path short.

Clean supply

Decouple every IC with ~100 nF close to its pins; add bulk capacitance near sensors with fans, lasers or heaters; give power-hungry sensors their own regulator so their current spikes do not reset the logger.

SECTION 11

I²C in practice

Sizing pull-ups, staying inside the capacitance budget, resolving address clashes, clock stretching on a Raspberry Pi, and recovering a hung bus.

Sizing the pull-up resistors

R_p is a compromise. Too small, and a device cannot pull the line below V_{OL} = 0.4 V with its rated sink current. Too large, and the RC rise through the bus capacitance C_b is slower than the mode allows. The rise time is measured between 30 % and 70 % of V_{DD}, so for an RC charge $t_r = R_p C_b \cdot \ln(0.7/0.3) = 0.8473 \cdot R_p C_b$.

$$R_p(min) = (V_{DD} - V_{OL,max}) / I_{OL} \qquad R_p(max) = t_r / (0.8473 \cdot C_b)$$

$V_{OL,max}$ = 0.4 V; I_{OL} = 3 mA (Sm, Fm) or 20 mA (Fm+); t_r = 1000 / 300 / 120 ns (Sm / Fm / Fm+). Source: NXP UM10204 §7.1; TI SLVA689.

DESIGN CHART · VDD = 3.3 V

Valid pull-up window versus bus capacitance

Sm · Fm · Fm+

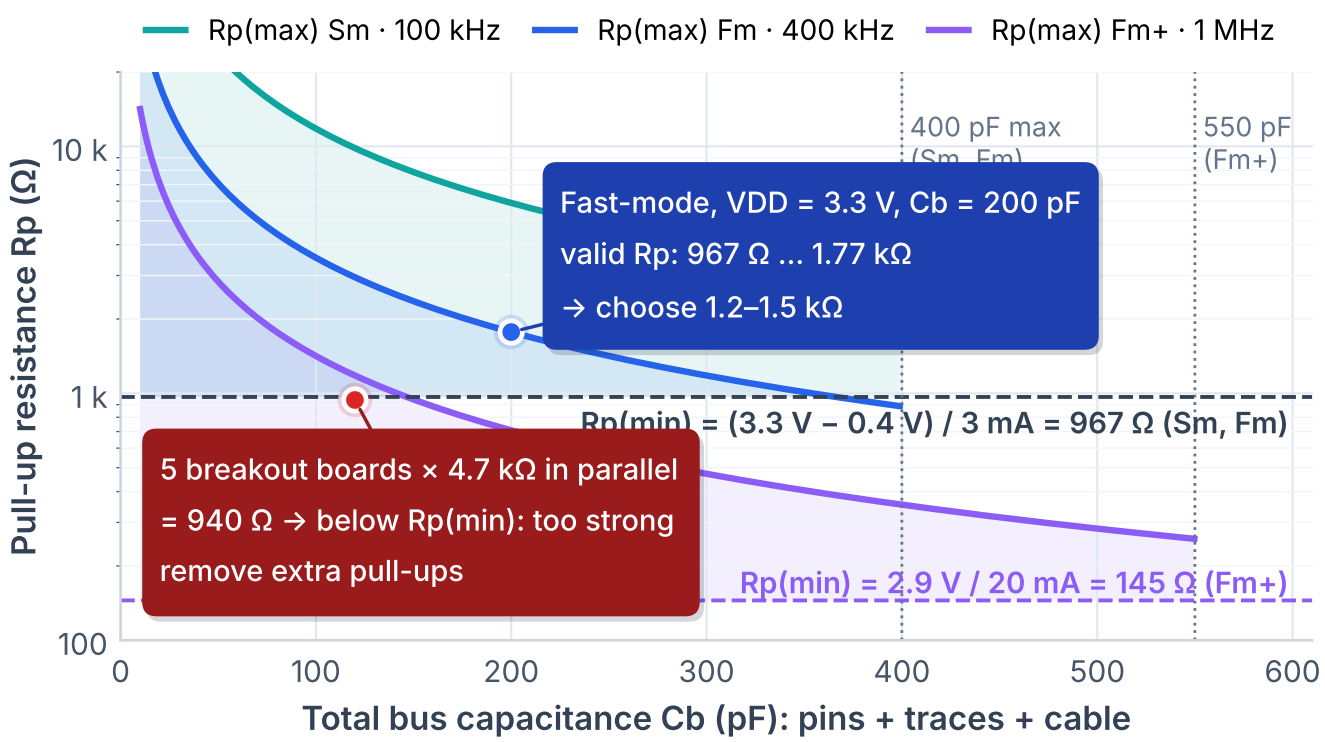


Figure 24. Any R_p between the dashed floor and the coloured curve is valid for that mode. Fast-mode at 3.3 V has no valid value beyond ≈ 366 pF; standard-mode reaches the 400 pF limit. Pull-ups on several breakout boards add in parallel.

UM10204 · SLVA689

Estimating C_b

Add up each device pin (UM10204 allows up to 10 pF), PCB traces (a few pF per 10 cm), connectors, and cable (check its datasheet; tens of pF per metre is common). Example: 4 sensors × 10 pF + 30 pF board + 2 m × 60 pF/m ≈ 190 pF, so Fast-mode needs R_p ≈ 1–1.8 kΩ.

Too many pull-ups

Breakout boards usually carry their own 4.7–10 kΩ pull-ups. Five boards with 4.7 kΩ give 940 Ω, below the 967 Ω minimum at 3.3 V: the bus may not reach a valid LOW. Remove or disable extra pull-ups (solder jumpers) so one pair remains.

Getting longer, or more, I²C

Problem	Options
Capacitance too high for the speed	Drop to 100 kHz (or lower); strengthen pull-ups within I_{OL}; add a rise-time accelerator (e.g. LTC4311) or a bus buffer that splits capacitance between segments.
Off-board cable of metres	Bus extenders (P82B715), differential I²C buffers over twisted pair (PCA9615), or move the sensor to UART/RS-485 if it offers one.
Two sensors with the same fixed address	Use a TCA9548A 1-to-8 switch (addresses 0x70–0x77): select a channel, then talk to the sensor behind it. Keep the mux off 0x76/0x77 if a BME280 shares the upstream bus.
Different voltage domains	Level shifter per segment (Section 10); each side keeps its own pull-ups.

Table 13. Common I²C scaling problems and standard fixes.

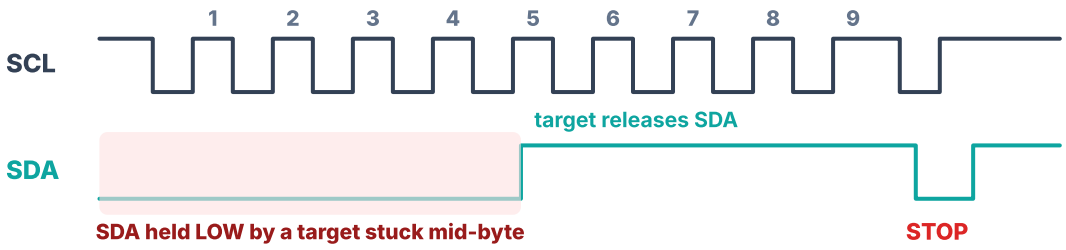
Clock stretching and the Raspberry Pi

The hardware I²C controller in the Raspberry Pi 1–4 (BCM283x/BCM2711) handles slave clock stretching poorly; the Linux driver flags it as not supporting stretching, and parts such as the SCD30 then return corrupt data or time out. Proven workarounds: lower the bus clock, use the bit-banged `i2c-gpio` overlay (which honours stretching), or use the sensor's alternative interface (the SCD30 also speaks Modbus RTU on UART).

/BOOT/FIRMWARE/CONFIG.TXT (RASPBERRY PI OS BOOKWORM)

```
# hardware I²C-1 at a conservative 50 kHz
dtparam=i2c_arm=on
dtparam=i2c_arm_baudrate=50000
# OR: a software I²C bus that tolerates clock stretching → /dev/i2c-3 on GPIO23/24
dtoverlay=i2c-gpio,bus=3,i2c_gpio_sda=23,i2c_gpio_scl=24
```

Recovering a hung bus



Bus clear (UM10204 §3.1.16): clock SCL up to nine times until SDA goes HIGH, then send a STOP.
Use it after a reset/brown-out interrupted a transfer and the bus 'hangs' with SDA LOW.

Figure 25. If the master resets in the middle of a read, a slave may still be driving a 0 and wait forever for clocks. Up to nine clock pulses walk it through the rest of its byte; then a STOP restores idle.

Linux bring-up

SHELL: IS THE SENSOR THERE?

```
$ i2cdetect -y 1          # scan bus 1
     0  1  2  3 ... 6  7
70: -- -- -- -- ... 76 --  # BME280
$ i2cget -y 1 0x76 0xD0
0x60                       # chip ID ✓
```

PYTHON (SMBUS2)

```
from smbus2 import SMBus
with SMBus(1) as bus:          # /dev/i2c-1
    cid = bus.read_byte_data(0x76, 0xD0)
    print(hex(cid))             # 0x60
# write reg, repeated START, read 1 byte
```

SENSIRION CRC-8 (SCD30, SPS30, SHT4X ...): POLYNOMIAL 0X31, INIT 0XFF

```
def crc8(data: bytes) -> int:
    crc = 0xFF
    for b in data:
        crc ^= b
        for _ in range(8):
            crc = ((crc << 1) ^ 0x31) & 0xFF if crc & 0x80 else (crc << 1) & 0xFF
    return crc
assert crc8(b"\xBE\xEF") == 0x92     # test vector from the SCD30 interface description
```

SECTION 12

UART in practice

Baud-rate accuracy, the timing budget, how long a reading really takes, turning a byte stream into trusted frames, and the Raspberry Pi's two UARTs.

Baud-rate error is arithmetic, not luck

A UART derives its bit clock by dividing a reference clock. With 16× oversampling and an integer divisor, actual baud = f_{clk} / (16 · round(f_{clk} / (16 · baud))). Some clock frequencies divide standard rates exactly (1.8432, 11.0592, 14.7456, 18.432 MHz, the "UART crystals"); round-number clocks often do not. Classic example: a 16 MHz ATmega328P (Arduino Uno) at 115200 baud runs at −3.5 % with normal-speed mode or +2.1 % with the double-speed (8×) mode. Many newer MCUs (e.g. STM32, ESP32, RP2040) use fractional dividers that keep the error far smaller.

COMPUTED HEATMAP · INTEGER DIVISOR, 16× OVERSAMPLING

Baud-rate error by clock frequency

● < 1 % ● 2–4 % ● > 4 %

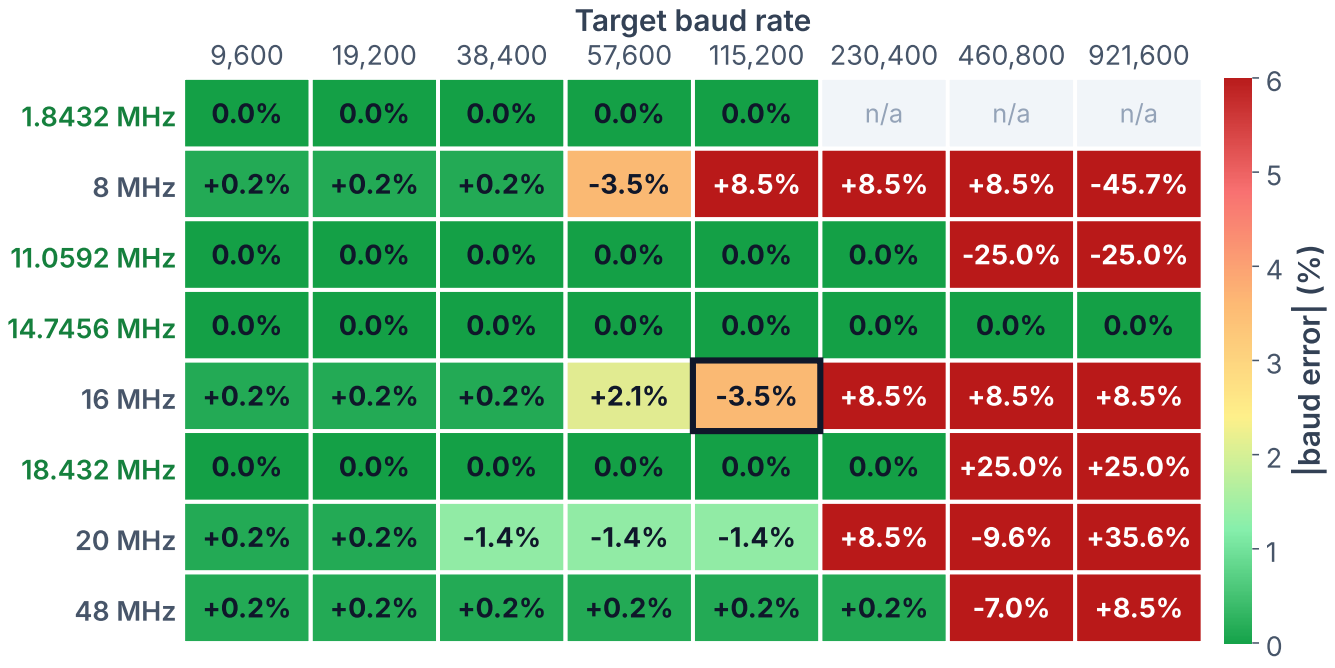


Figure 26. Green row labels are "UART crystals". The outlined cell is the Arduino Uno case (16 MHz, 115200 baud, U2X = 0). "n/a": divisor would be below 1. **computed**

Practical rule

The chart below shows where the budget comes from. Keep each end within about ±1 % of nominal (half the conservative ±2 % budget). A ceramic resonator (±0.5 % plus drift) is usually adequate; an MCU's internal RC oscillator may not be, especially over outdoor temperature swings, unless it is factory-trimmed and specified for UART use.

TIMING BUDGET · 8N1 FRAME

How clock mismatch walks the sample point off the bit centre

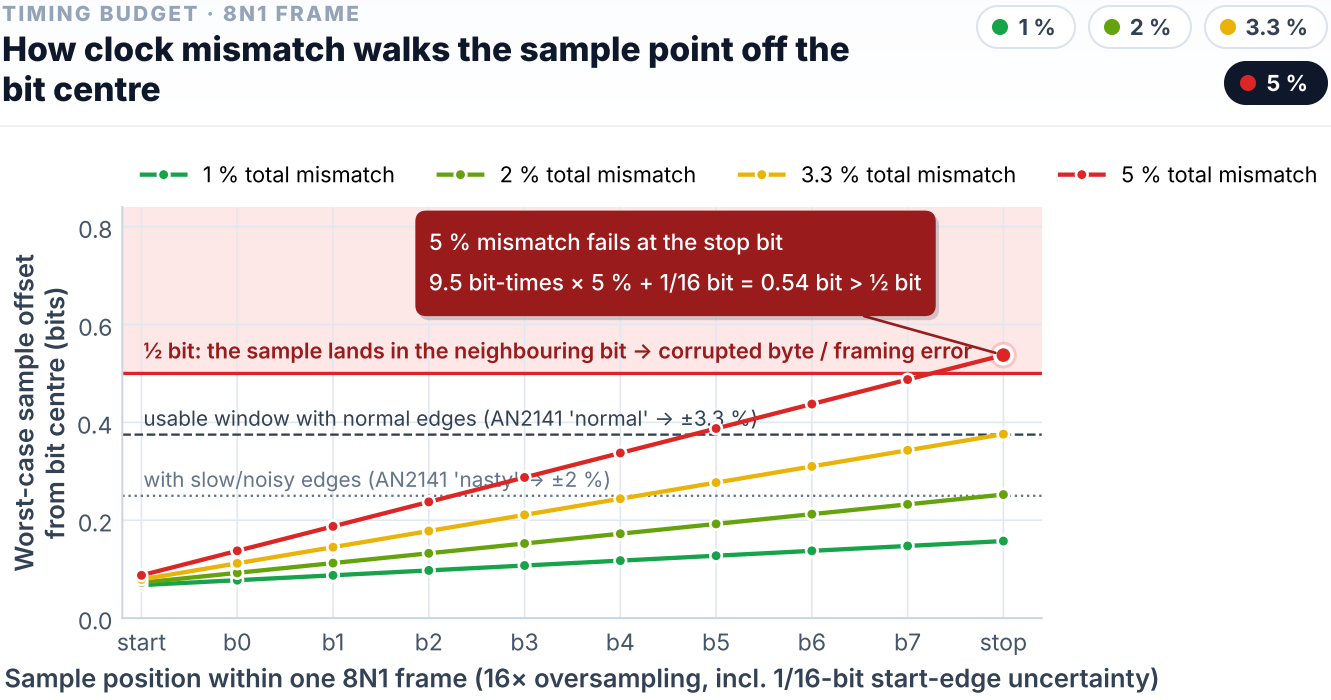


Figure 27. Offset at sample k = (k + ½) × mismatch + 1/16 bit. The stop bit is sampled 9.5 bit-times after the start edge, so it fails first. Analog Devices/Maxim derive a total budget of ±3.3 % (clean edges) or ±2 % (slow, noisy edges), to be shared between transmitter and receiver. AN 2141

How long does a reading take on the wire?

COMPUTED · IDEAL WIRE TIME

Moving 32 bytes of sensor data

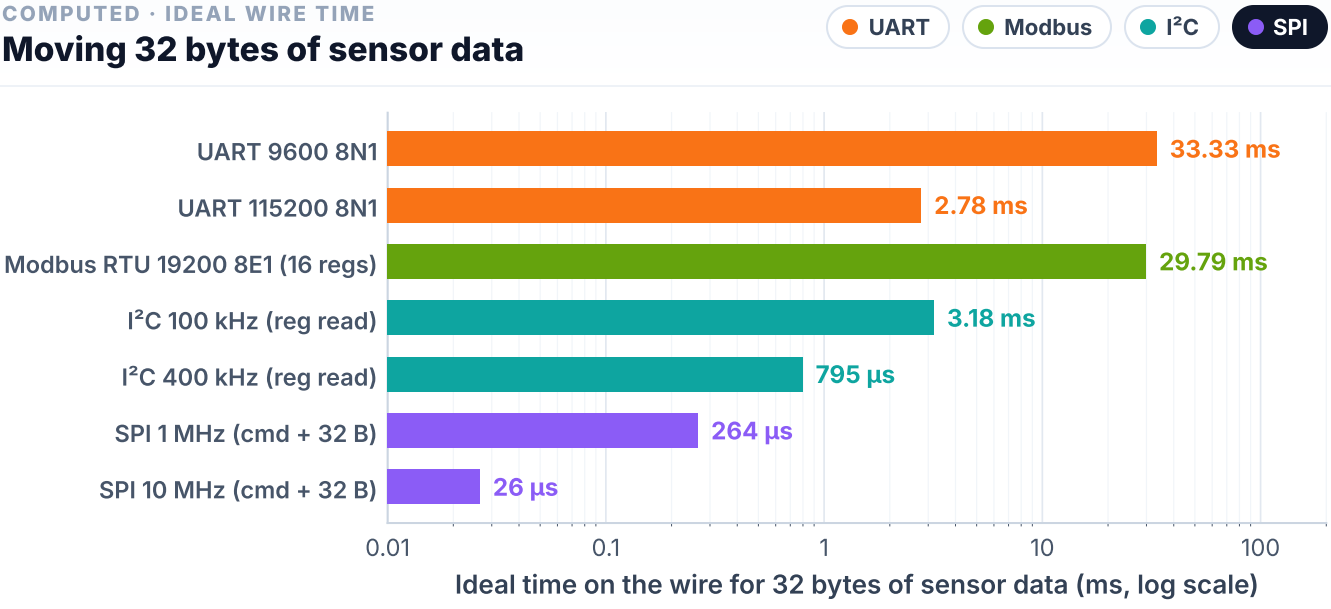


Figure 28. UART: 10 bits/byte. Modbus: 8-byte request + 37-byte reply at 11 bits/char, plus two 3.5-character gaps. I²C: register-pointer write, repeated START, 32 data bytes (≈ 318 bit-times). SPI: one command + 32 bytes. Real devices add processing and turnaround time. computed

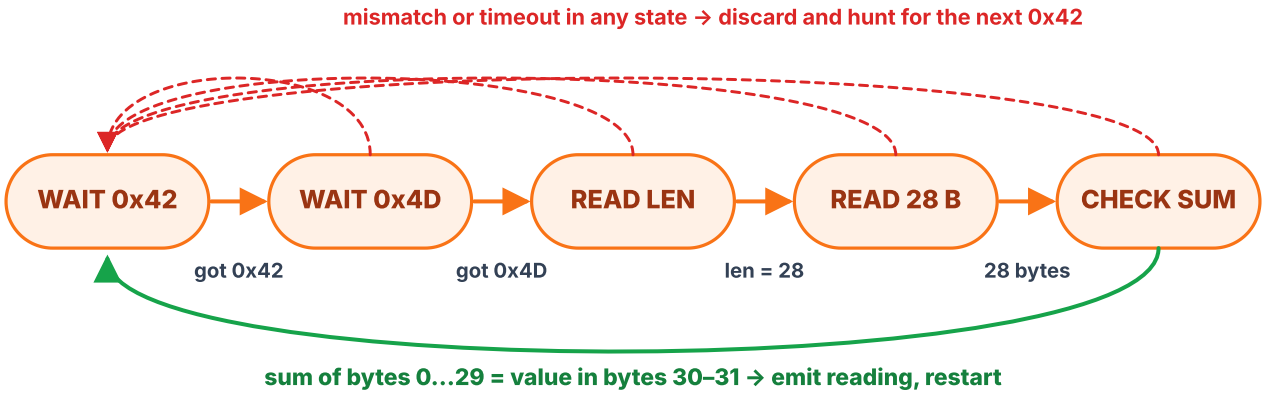
From byte stream to trusted frames

A serial port gives you bytes with no boundaries. Binary sensors therefore wrap each reading in a frame with a header, a length and a check value. The PMS5003 frame is a good model:

PMS5003 frame (32 bytes, 16-bit values high byte first)

42 start 1	4D start 2	00 1C length = 28	PM1 CF=1	PM2.5 CF=1	PM10 CF=1	PM1 atm.	PM2.5 atm.	PM10 atm.	>0.3 /0.1 L	>0.5	>1.0
>2.5	>5.0	>10 µm	rsvd	sum checksum							

Checksum = 16-bit sum of the 30 preceding bytes. PM values in µg/m³; counts are particles larger than the stated diameter per 0.1 L of air (Plantower data manual).



A serial port delivers bytes with no frame boundaries: find the header, check the length, verify the checksum, then trust the payload. Same pattern for SHDLC, NMEA and Modbus.

Figure 29. A resynchronising parser. Never assume the first byte you read is the start of a frame: the logger may start mid-frame, and noise can drop bytes.

PYTHON (PYSERIAL): READ ONE VERIFIED PMS5003 FRAME

```python
import serial, struct

def read_pms5003(port="/dev/serial0"):
    with serial.Serial(port, 9600, timeout=2) as s:   # pyserial defaults to 8N1
        while True:
            if s.read(1) != b"\x42" or s.read(1) != b"\x4d":
                continue                               # hunt for the header
            body = s.read(30)                          # length + 13 values + checksum
            if len(body) != 30:
                continue                               # timeout: resynchronise
            length, *vals, chk = struct.unpack(">15H", body)
            if length == 28 and (0x42 + 0x4D + sum(body[:28])) & 0xFFFF == chk:
                return {"pm1": vals[3], "pm25": vals[4], "pm10": vals[5]}  # atmospheric values
```

Flow control

Hardware flow control (RTS/CTS) lets a receiver pause the sender before its buffer overflows. Software flow control (XON 0x11 / XOFF 0x13 in the data stream) cannot be used with binary data, because those byte values occur naturally. Most sensors use neither: read promptly and let the OS buffer.

Reading habits

Open ports in raw mode (pyserial does); always set a timeout; never use `readline()` on binary protocols; log raw frames that fail validation, with a timestamp, to diagnose problems later.

The Raspberry Pi's two UARTs

On the Pi 3, 4 and Zero W the full PL011 UART (`/dev/ttyAMA0`) is connected to Bluetooth by default, and GPIO14/15 get the "mini UART" (`/dev/ttyS0`), whose baud rate is derived from the variable VPU core clock. `/dev/serial0` always points to whichever UART is on GPIO14/15. Also disable the Linux serial console on that port, or a login shell will read your sensor's bytes.

/BOOT/FIRMWARE/CONFIG.TXT

```
enable_uart=1          # enable GPIO14/15 UART; also locks the core clock for the mini UART
dtoverlay=disable-bt   # optional: move the PL011 to GPIO14/15 (then: sudo systemctl disable
hciuart)
# sudo raspi-config → Interface Options → Serial Port: login shell = No, hardware = Yes
```

SECTION 13

SPI in practice

MHz

Mode mismatches, timing over wires, shared MISO, chip-select discipline and Linux spidev.

Symptoms of the wrong mode

Values exactly 2× or ½ of expected (data shifted by one bit), a correct ID one time in two, or first bit always wrong. Read the timing diagram in the datasheet, not just a "mode" number; some datasheets define CPHA differently.

Chip-select discipline

Hold CS LOW for an entire multi-byte command; some parts need CS to go HIGH between commands. Put a pull-up on every CS line so slaves stay deselected while the MCU boots or resets.

Shared MISO
Every unselected slave must float MISO (high impedance). A part that keeps driving it corrupts the bus for everyone; give it a tri-state buffer or its own SPI bus.

Wires and ringing
Fast SPI edges ring on long wires. Keep SPI on the board or on short ribbons with a ground per signal; a 22–33 Ω series resistor at each driver damps reflections; lower the clock first when in doubt.

How fast over a cable? A timing bound

In mode 0, slave data launched on a falling SCLK edge must reach the master before the next rising edge, half a period later. The clock travels out, the slave reacts (clock-to-output t_{CO}) and the data travels back, so

$$f_{SCLK,max} \approx 1 / [\ 2 \cdot (2\ t_{pd} + t_{CO} + t_{SU})\ ]$$

t_{pd} = one-way cable delay (≈ 5 ns/m), t_{CO} = slave clock-to-output, t_{SU} = master input setup time (5 ns assumed below).

COMPUTED · TIMING ONLY

Maximum SPI clock versus cable length

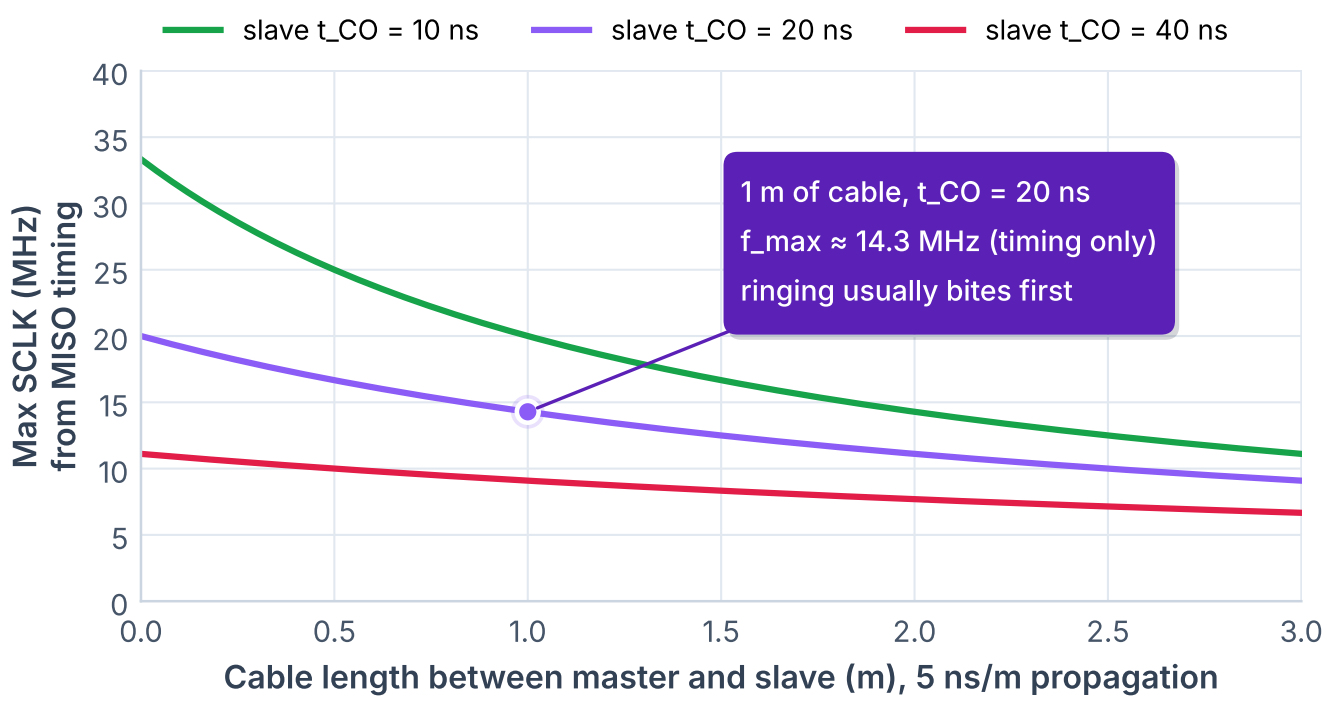


Figure 30. This is an upper bound from round-trip delay alone. Ringing, crosstalk and slow edges usually limit off-board SPI well below it. **computed**

PYTHON (SPIDEV): READ THE BME280 CHIP ID OVER SPI

```
import spidev
spi = spidev.SpiDev()
spi.open(0, 0)                    # /dev/spidev0.0 → SPI0, CE0
spi.mode = 0b00                   # CPOL=0, CPHA=0 (BME280 also accepts 0b11)
spi.max_speed_hz = 1_000_000
rx = spi.xfer2([0xD0, 0x00])      # register 0xD0 with bit7=1 (read) + one dummy byte
print(hex(rx[1]))                  # 0x60; rx[0] is clocked in while the command goes out
spi.close()
```

Daisy chains
Some devices (shift registers, LED drivers, some ADCs) can be chained: MISO of one feeds MOSI of the next, all share one CS, and N devices need N × 8 clocks per update. Only use it if every device in the chain supports it.

SECTION 14

USB, USB-serial & RS-232 in practice

Stable port names, hidden latency in USB bridges, power budgets, unplug-proof loggers, and RS-232 field habits.

Give every sensor a stable name

`/dev/ttyUSB0` and `/dev/ttyUSB1` are numbered in plug-in (enumeration) order, which can change on every reboot. Use the persistent links in `/dev/serial/by-id/`, or write a udev rule keyed on vendor ID, product ID and serial number:

```
# /etc/udev/rules.d/99-sensors.rules   (FTDI FT232R = 0403:6001, CP2102 = 10c4:ea60, CH340 = 1a86:7523)
SUBSYSTEM=="tty", ATTRS{idVendor}=="0403", ATTRS{idProduct}=="6001", ATTRS{serial}=="A10KXYZ1", SYMLINK+="rs485_met"
# then: sudo udevadm control --reload && sudo udevadm trigger   → /dev/rs485_met
```

The latency you did not ask for

USB bridges do not forward every byte immediately. An FTDI chip sends its buffer to the host when the buffer fills (62 data bytes + 2 status bytes), when an event character arrives, when a modem-status line changes, or when its **latency timer** expires (default 16 ms, adjustable 1–255 ms). A short Modbus reply therefore reaches your program up to ~16 ms late, and gaps between bytes are invisible to host software, so host-side t3.5 timing cannot be measured precisely.

```
$ cat /sys/bus/usb-serial/devices/ttyUSB0/latency_timer
16
$ echo 1 | sudo tee /sys/bus/usb-serial/devices/ttyUSB0/latency_timer   # 1 ms (ftdi_sio driver)
```

Power budget

A USB 2.0 port gives at most 500 mA after enumeration. Several PM sensors (fans, lasers) plus a GNSS plus an RS-485 dongle can exceed it; brown-outs then reset devices and they re-enumerate under new names. Use a powered hub or a separate 5 V supply, and short, thick cables.

Survive unplugs

Wrap reads in exception handlers; on `SerialException`, close, wait, and reopen by stable name. Add a watchdog that restarts the logger if no valid frame has arrived for N intervals. Beware charge-only USB cables (no data pairs).

Cable length and isolation

Passive USB 2.0 cables are limited to about 5 m; use active extenders or move to RS-485 for distance. USB isolators (e.g. ADuM3160 for low/full speed) break ground loops between a PC and grounded field equipment.

RS-232 field habits

Loopback test first: short pins 2 and 3 on the adapter; characters you type must echo. Know which "USB-serial" adapter you have: true RS-232 (±V on a DE-9) and logic-level "TTL" adapters (3.3/5 V header pins) are not interchangeable.

Handshake lines that block data

Some instruments will not transmit until they see DTR or CTS asserted, while some USB adapters toggle DTR/RTS when a port opens (this is how an Arduino gets reset). If a device is silent or resets when you connect, check the handshake settings (pyserial: `rtscts`, `dsrdtr`, `dtr`, `rts`).

SECTION 15

RS-485 & Modbus RTU in practice

Data rate versus length, termination, fail-safe biasing, stubs, grounding, direction control and Modbus timing.

RS-485 links that fail in the field usually fail for one of five reasons: too long for the bit rate, missing or extra terminations, no fail-safe bias, long stubs, or ground/common-mode problems. Direction control and Modbus timing account for most of the rest. Each is covered below.

Termination and stubs

A 120 Ω resistor at each physical end matches the cable's characteristic impedance so signals are absorbed instead of reflected. Terminate only the two ends, never every node. A **stub** (the branch

from the trunk to a transceiver) must be electrically short compared with the driver's rise time t_r:

$$L_{stub} \le t_r \cdot v \cdot c / 10 \quad \text{with } v \approx 0.78 \text{ (velocity factor) and } c = 3\times10^8 \text{ m/s}$$

t_r = 100 ns → 2.3 m (7.7 ft) · 250 ns → 5.9 m (19 ft) · 500 ns → 11.7 m (38 ft). Slew-limited (slow-edge) transceivers allow longer stubs and radiate less. Source: TI SLLA272.

How fast, how far?

DESIGN CHART · 24 AWG TWISTED PAIR

RS-485 data rate versus cable length

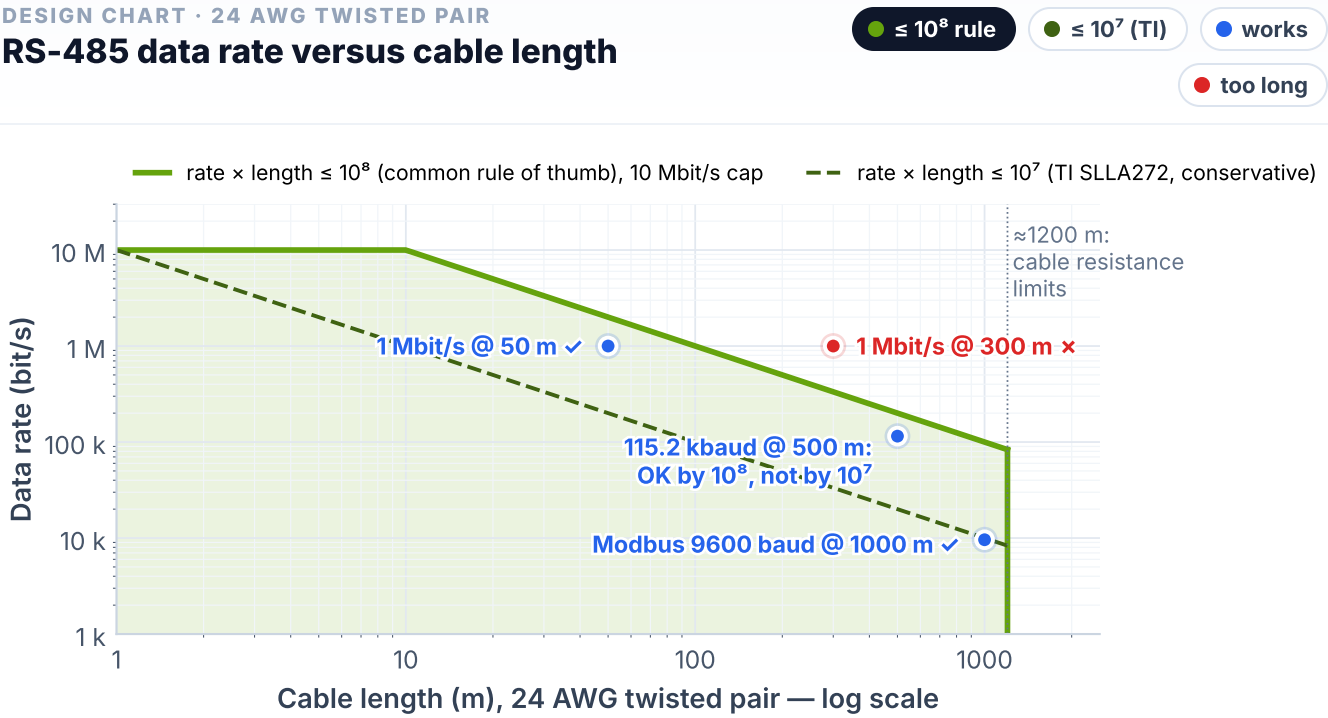


Figure 31. At short lengths the transceiver limits speed; then cable loss and edge degradation scale roughly as length × rate; beyond ≈ 1200 m DC resistance dominates. Typical meteorological Modbus links (9600–19200 baud) have large margins.

TIA-485-A · TI SLLA272

Fail-safe biasing

Between frames no driver is on. With terminations the pair then sits near 0 V differential, inside the ±200 mV undefined zone, and receivers may output noise that looks like start bits. One pull-up on D+ and one pull-down on D− (placed at one node only) hold the idle bus at a valid level:

$$V_{AB,idle} = V_{CC} \cdot R_{eq} / (2\,R_b + R_{eq}) \quad R_{eq} = 120\ \Omega \parallel 120\ \Omega \parallel (12\ k\Omega / N_{UL})$$

Fully loaded (32 unit loads): R_{eq} = 51.7 Ω. For V_{AB} ≥ 250 mV (200 mV + 50 mV noise margin): R_b ≤ 466 Ω at 4.75 V, ≤ 298 Ω at 3.135 V.

DESIGN CHART

Idle differential voltage versus bias resistor

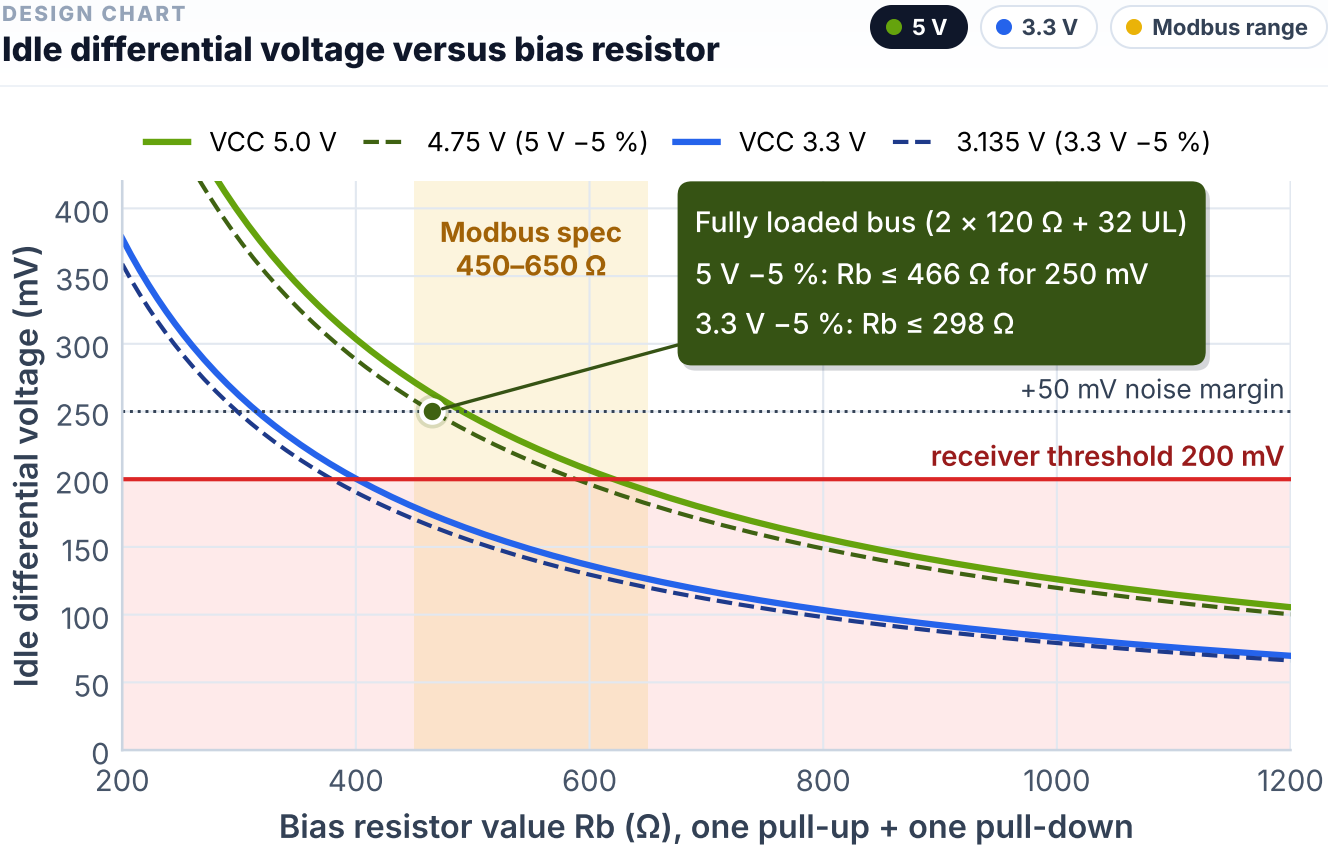


Figure 32. The Modbus serial-line specification suggests 450–650 Ω bias resistors; at 5 V that is adequate for lightly loaded buses but marginal with 32 unit loads, and too weak at 3.3 V. Alternatively, use transceivers with "true fail-safe" receivers that read 0 V as idle.

Modbus V1.02 · computed

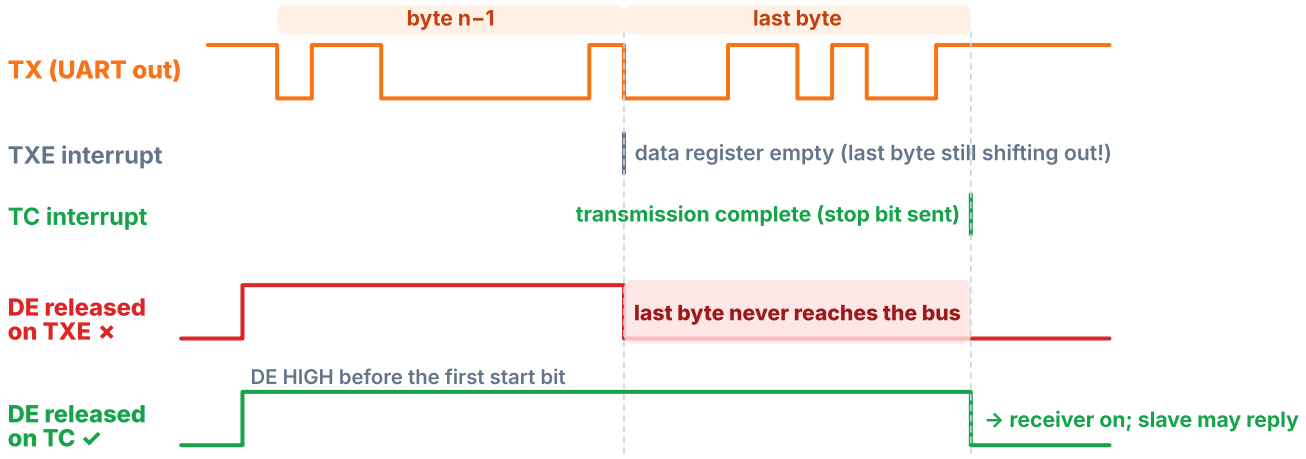
Grounding

Each receiver needs its inputs within −7…+12 V of its own ground. Run a common (reference) conductor with the pair; the Modbus serial-line spec includes one. Between separately earthed buildings or masts, use isolated transceivers rather than tying distant earths together (TI cautions that hard remote-ground connections invite ground-loop currents).

Surge and lightning

Outdoor runs need TVS protection matched to the bus range (e.g. SM712, −7/+12 V), shielded twisted pair, and isolation for long masts. Protection is cheaper than replacing a weather station.

Direction control in half-duplex



Half-duplex RS-485: the driver must stay enabled until the last stop bit has physically left the UART.

Figure 33. Releasing the driver on the "transmit buffer empty" interrupt cuts off the last byte; wait for "transmission complete". USB-RS485 adapters (e.g. FTDI chips with the TXDEN pin) and auto-direction transceivers do this in hardware; Linux UARTs can do it in the kernel's RS-485 mode.

Modbus RTU essentials

Rule	Value (MODBUS over Serial Line V1.02)
Character format	1 start, 8 data (LSB first), 1 parity, 1 stop = 11 bits. Default parity **even**; with no parity, use **2 stop bits** (many devices still ship 8N1: match the manual).
Baud rates	9600 and 19200 required; 19200 is the default.
Addresses	0 = broadcast (no reply), 1–247 = slaves, 248–255 reserved.
Frame gap t3.5	≥ 3.5 character times of silence: 4.01 ms at 9600, 2.01 ms at 19200; fixed 1.75 ms above 19200.
Inter-character t1.5	A gap > 1.5 characters inside a frame invalidates it (fixed 750 µs above 19200).
Frame size · CRC	≤ 256 bytes; CRC-16 appended low byte first.
Bus	≥ 32 devices without repeater; terminate both ends; bias resistors 450–650 Ω.

Table 14. Modbus RTU rules that matter for wiring and timing; times computed for 11-bit characters.

Code	Function	Typical use on sensors
03	Read holding registers	most measurements and settings
04	Read input registers	read-only measurements (vendor-dependent)
06	Write single register	change slave ID, baud rate, averaging
16 (0x10)	Write multiple registers	multi-word settings, calibration constants

Table 15. The four function codes you will use most.

PYTHON (MINIMALMODBUS)

```python
import minimalmodbus, serial
m = minimalmodbus.Instrument("/dev/rs485_met", 1)
m.serial.baudrate = 19200
m.serial.parity = serial.PARITY_EVEN
m.serial.timeout = 0.5                # s
t = m.read_register(0, number_of_decimals=1,
                    functioncode=3)     # 250 → 25.0
```

MODBUS CRC-16 BY HAND

```python
def crc16_modbus(data: bytes) -> bytes:
    crc = 0xFFFF
    for b in data:
        crc ^= b
        for _ in range(8):
            crc = (crc >> 1) ^ 0xA001 if crc & 1 else crc >> 1
    return crc.to_bytes(2, "little")
assert crc16_modbus(bytes([1,3,0,0,0,2])) == b"\xC4\x0B"
```

SECTION 16

Data integrity across the stack

CRC

Every link loses bits occasionally. What decides data quality is whether the error is detected before a bad value enters your dataset.

Check	Where	Detects	Misses
Parity (1 bit)	UART characters (8E1, 7E1)	Any odd number of flipped bits in one character.	Any even number (e.g. 2 adjacent bits hit by a spike).
ACK / NACK	I²C bytes	Absent, busy or wrong-address device.	Corrupted bits inside a byte.
XOR checksum (8 bit)	NMEA 0183 sentences	All single-bit errors; most random errors.	Two errors in the same bit column; reordered characters.

Check	Where	Detects	Misses
Sum checksum (16 bit)	PMS5003 frames	Most random errors.	Reordered bytes; errors that cancel in the sum.
CRC-8, poly 0x31	Sensirion I²C words	All single-bit, all odd-count, all bursts ≤ 8 bits.	Some longer or multi-burst patterns (≈ 1 in 256 random).
CRC-16, poly 0x8005	Modbus RTU, USB data	All single-bit, all odd-count, all bursts ≤ 16 bits.	≈ 1 in 65 536 random corruptions.

Table 16. Error-detection mechanisms you will meet. A CRC of degree n detects every burst of length ≤ n (Peterson & Brown, 1961); both polynomials above contain the factor (x + 1), so every odd number of bit errors is also caught.

Why CRC beats a sum

A CRC treats the message as a polynomial over GF(2) and keeps the remainder after division by a generator polynomial. Its guarantees follow from algebra, not from luck. Koopman & Chakravarty (2004) showed that the choice of polynomial strongly affects detection for the short messages typical of embedded networks; standard polynomials are not always the best, but they are what your sensors use.

What to do in the logger

Validate every frame (length, header, CRC/checksum, plausible range). Count failures per sensor per hour as a health metric; a rising error rate is an early warning of a failing cable, connector or power supply. Store raw frames for failed checks; never silently "fix" them.

Detection is not correction

None of these links corrects errors on its own. USB retries automatically; for the others, your software decides: re-request (Modbus, I²C), wait for the next frame (streaming UART sensors), or mark the sample as missing.

SECTION 17

NMEA 0183 & SDI-12

Two serial formats common on meteorological and hydrological instruments, both built on UART-style characters.

NMEA 0183 (GNSS, marine/weather instruments)

ASCII sentences: `$` + talker/sentence ID + comma-separated fields + `*` + two hex checksum digits + CR LF; at most 82 characters. Standard rate 4800 baud 8N1 on an RS-422-style electrical layer (NMEA-0183HS: 38 400 baud); many GNSS modules simply use 9600 baud on a logic-level UART. Checksum: XOR of every character between `$` and `*`.

`$GPGLL` `4916.45,N` `12311.12,W` `225444,A` `*31` XOR

SDI-12 (environmental & hydrological sensors)

One data wire plus ground and 12 V power. 1200 baud, 7 data bits, even parity, 1 stop bit (7E1), with inverted 5 V logic (a 1 is near 0 V). Up to 62 sensors share the line, each addressed by one character (0–9, a–z, A–Z). The logger wakes sensors with a break, then sends commands such as `0M!` (start measurement) and `0D0!` (send data); sensors answer in ASCII.

```
from functools import reduce
def nmea_ok(line: str) -> bool:
    body, _, cs = line.strip()[1:].partition("*")          # drop '$' or '!'
    return f"{reduce(lambda a, c: a ^ ord(c), body, 0):02X}" == cs.upper()
assert nmea_ok("$GPGLL,4916.45,N,12311.12,W,225444,A*31")
```

SECTION 18

Bring-up & troubleshooting

Work from the physical layer upward. Measure before you change code.

Multimeter

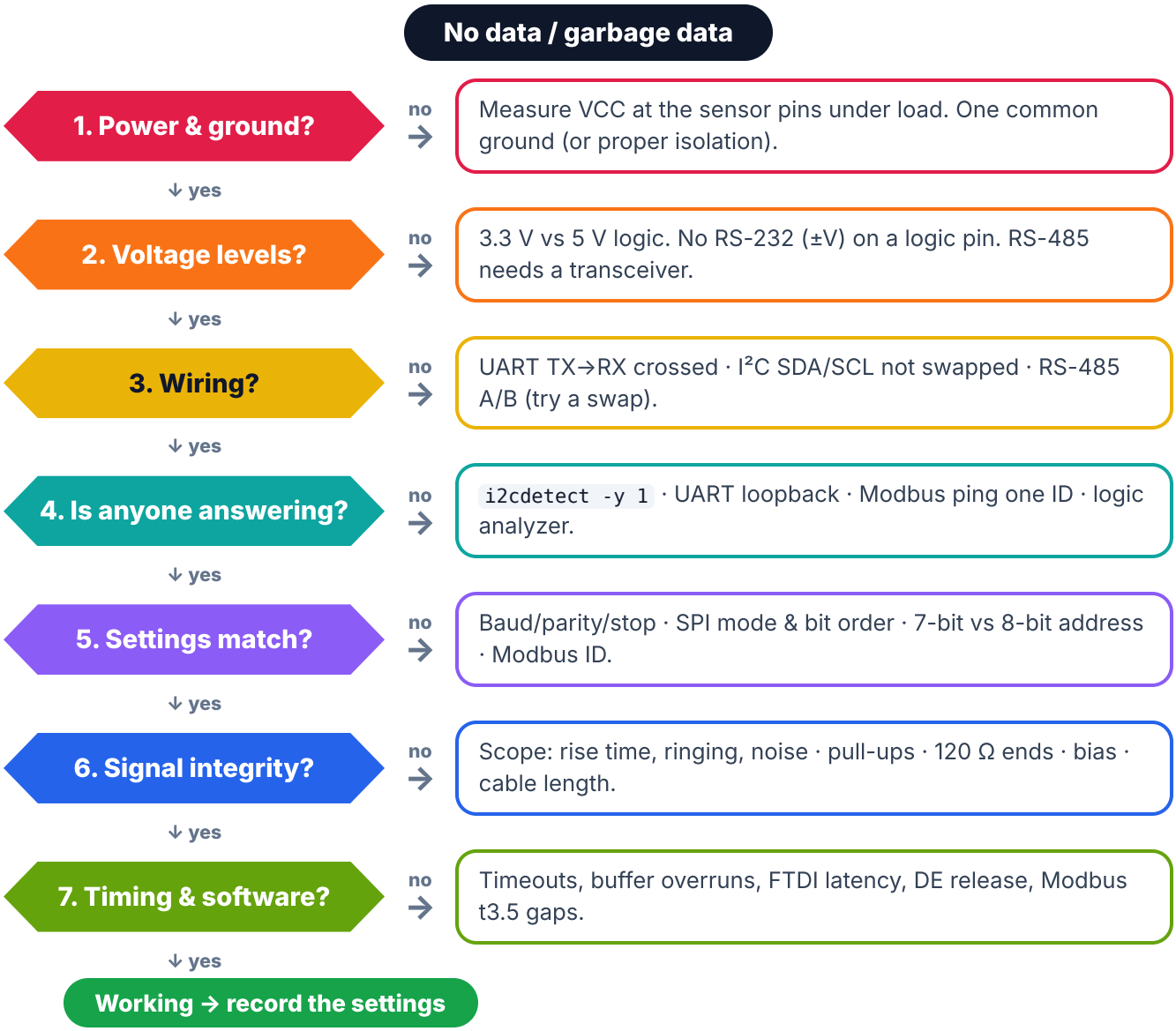
Supply at the sensor pins under load; idle levels ($I^2C \approx V_{DD}$, UART TX HIGH, RS-485 idle $V_{AB} > 200$ mV); continuity; termination (≈ 60 Ω across a powered-off, correctly terminated pair).

Logic analyzer

Inexpensive and decisive. Protocol decoders (e.g. sigrok/PulseView) show I^2C addresses and ACKs, UART bytes with framing errors, SPI transfers and Modbus frames.

Oscilloscope

Rise times, ringing, noise, voltage levels. For RS-485 use two channels and the A–B math trace (or a differential probe).



Troubleshooting order. Power, levels, wiring, presence, settings, signal integrity, then software timing. Most faults are found in the first three steps.

Symptom	Most likely causes
Random characters at a steady rate (UART)	Baud mismatch; inverted signal (RS-232 into a logic pin); wrong data bits/parity.
Correct bytes but some missing	Overrun: reading too slowly, no FIFO/DMA, or other code blocking; console attached to the port.
`i2cdetect` **shows nothing**	No pull-ups; SDA/SCL swapped; sensor unpowered; wrong bus number; I²C not enabled.
`i2cdetect` **shows every address**	SDA held LOW (stuck slave, short, back-powered chip): every probe looks acknowledged. Run a bus clear.

Symptom	Most likely causes
I²C works on the bench, fails on a cable	Capacitance too high for the pull-ups/speed; crosstalk; slow to 100 kHz or less, re-size R_p, buffer or change interface.
SPI values off by ×2 or ÷2	Wrong CPOL/CPHA; bit order; CS toggled mid-command.
SPI always reads 0x00 or 0xFF	CS not asserted; MISO open; slave unpowered or in a different mode.
RS-485 completely silent	A/B swapped; wrong ID, baud or parity; DE never enabled; no common reference.
RS-485 works with one node, fails with several	Missing/duplicate termination or bias; duplicate Modbus IDs; long stubs; too many unit loads.
Modbus timeouts now and then	Response timeout too short; DE released early; USB latency timer; t3.5 gaps violated by the sender.
USB sensor disappears or renames	Brown-out on a busy port; loose/charge-only cable; EMI; use stable names and a powered hub.

Table 17. Symptom → cause lookup.

SECTION 19

Deployment checklist & formula sheet

Tick every box before a node leaves the lab; keep the formulas next to the bench.

Before deployment

- ☐ Every interface setting (baud, parity, stop bits, SPI mode, I²C speed, Modbus ID) written on the node's record sheet.
- ☐ All logic levels checked: no 5 V outputs into 3.3 V-only pins; RS-232 only through a transceiver.
- ☐ Stable device names (udev rules / by-id); serial console disabled on sensor UARTs.
- ☐ Power budget measured with all fans/heaters running; powered hub or separate supply where needed.
- ☐ TVS/ESD protection on every external connector; shield and earth strategy documented.
- ☐ I²C bus-clear routine at logger start-up; sensor ID registers read and logged at boot.
- ☐ One pull-up pair per I²C bus segment; rise time checked on a scope at the deployed cable length.
- ☐ RS-485: 120 Ω at both ends only; one bias pair; idle V_{AB} > 200 mV measured; common conductor run.
- ☐ Every frame validated (CRC/checksum/length); failures counted and logged with raw bytes.
- ☐ Logger survives unplug/replug and power cycles (exception handling, watchdog, auto-restart service).
- ☐ Timestamps from a disciplined clock (NTP or GNSS PPS); time zone stated as UTC in the data.
- ☐ 24-hour soak test on the bench with the final cables, checking error counters before field install.

Formula sheet

Quantity	Formula	Example
Bit time	$T_{bit} = 1 / baud$	9600 → 104.17 µs
UART byte rate	$baud / (1 + data + parity + stop)$	9600 8N1 → 960 B/s
UART baud error	$f/(16 \cdot round(f/(16 \cdot baud)))/baud - 1$	16 MHz, 115200 → −3.5 %
UART timing budget	$(k + \frac{1}{2}) \cdot \Delta + 1/16 < window$	total ±3.3 % (clean) / ±2 % (noisy)
I²C pull-up range	$(V_{DD}-0.4)/I_{OL} \le R_p \le t_r/(0.8473 \cdot C_b)$	Fm, 3.3 V, 200 pF → 967 Ω–1.77 kΩ
I²C transfer time	≈ 9 bit-times per byte + start/stop	32-byte read at 400 kHz ≈ 0.8 ms
SPI clock bound	$1 / [2 \cdot (2t_{pd} + t_{CO} + t_{SU})]$	1 m, t_{CO} = 20 ns → 14 MHz
RS-485 rate × length	$R \cdot L \le 10^8$ ($\le 10^7$ conservative)	1200 m → ≈ 83 kbit/s
RS-485 stub length	$L \le t_r \cdot 0.78 \cdot c / 10$	t_r = 100 ns → 2.3 m
RS-485 idle bias	$V_{CC} \cdot R_{eq}/(2R_b+R_{eq}) \ge 200$ mV	5 V, 32 UL → $R_b \le 466$ Ω
Modbus frame gap	$t3.5 = 3.5 \times 11 / baud$	9600 → 4.01 ms; > 19200 → 1.75 ms
RS-232 cable estimate	$L \approx 2500$ pF / C_{cable}	at 50 pF/ft ≈ 50 ft (15 m)

Table 18. Formulas used in this guide.

SOURCES

References

Primary specifications and manufacturer documents for every number in this guide, plus peer-reviewed literature.

Standards and specifications

[1] NXP Semiconductors. *UM10204: I²C-bus specification and user manual*, Rev. 7.0, 1 October 2021.

[2] TIA. *TIA-232-F: Interface Between Data Terminal Equipment and Data Circuit-Terminating Equipment Employing Serial Binary Data Interchange*, 1997.

[3] TIA. *ANSI/TIA/EIA-485-A: Electrical Characteristics of Generators and Receivers for Use in Balanced Digital Multipoint Systems*, 1998 (reaffirmed 2012).

[4] USB Implementers Forum. *Universal Serial Bus Specification, Revision 2.0*, 2000.

[5] Modbus Organization. *MODBUS over Serial Line: Specification and Implementation Guide*, V1.02, 2006.

[6] National Marine Electronics Association. *NMEA 0183 Interface Standard*.

[7] SDI-12 Support Group. *SDI-12: A Serial-Digital Interface Standard for Microprocessor-Based Sensors*, Version 1.4.

Application notes

[8] Texas Instruments. *The RS-485 Design Guide*, SLLA272D, 2021.

[9] Texas Instruments. *I²C Bus Pullup Resistor Calculation*, SLVA689.

[10] Texas Instruments. *Interface Circuits for TIA/EIA-232-F*, SLLA037A.

[11] Analog Devices (Maxim Integrated). *Determining Clock Accuracy Requirements for UART Communications*, Application Note 2141, 2003.

[12] Analog Devices. *Introduction to SPI Interface*, Analog Dialogue.

[13] FTDI. *AN232B-04: Data Throughput, Latency and Handshaking*.

[14] Philips Semiconductors. *AN97055: Bi-directional level shifter for I²C-bus and other systems*, 1997.

[15] Raspberry Pi Ltd. *Raspberry Pi documentation: configuration, config.txt and UARTs*.

Datasheets and interface descriptions

[16] Bosch Sensortec. *BME280 Combined humidity and pressure sensor*, datasheet.

[17] Sensirion. *Interface Description Sensirion SCD30 Sensor Module*.

[18] Sensirion. *SPS30 Particulate Matter Sensor*, datasheet.

[19] Plantower. *PMS5003 series data manual*.

[20] Alphasense. *OPC-N3 Optical Particle Counter*, technical documentation; community driver py-opc-ng.

Peer-reviewed literature

[21] Leens, F. (2009). An introduction to I²C and SPI protocols. *IEEE Instrumentation & Measurement Magazine*, 12(1), 8–13. https://doi.org/10.1109/MIM.2009.4762946

[22] Peterson, W. W., & Brown, D. T. (1961). Cyclic codes for error detection. *Proceedings of the IRE*, 49(1), 228–235. https://doi.org/10.1109/JRPROC.1961.287814

[23] Koopman, P., & Chakravarty, T. (2004). Cyclic redundancy code (CRC) polynomial selection for embedded networks. *Proc. International Conference on Dependable Systems and Networks (DSN 2004)*, 145–154. https://doi.org/10.1109/DSN.2004.1311885

[24] Si, M., Xiong, Y., Du, S., & Du, K. (2020). Evaluation and calibration of a low-cost particle sensor in ambient conditions using machine-learning methods. *Atmospheric Measurement Techniques*, 13, 1693–1707. https://doi.org/10.5194/amt-13-1693-2020

[25] Balagopal, G., Wijeratne, L., Waczak, J., … Lary, D. J. (2025). Calibration of low-cost LoRaWAN-based IoT air quality monitors using the Super Learner ensemble: A case study for accurate particulate matter measurement. *Sensors*, 25(5), 1614. https://doi.org/10.3390/s25051614

Why the sensor papers are here

The links in this guide deliver numbers; whether those numbers are accurate is a separate question. Low-cost PM sensors such as the PMS5003 need field calibration against reference instruments, with humidity a major factor (Si et al., 2020); the UT Dallas group's LoRaWAN nodes in Dallas–Fort Worth were calibrated with machine-learning ensembles against a research-grade instrument (Balagopal et al., 2025). Clean, validated, well-timestamped data from the links is the prerequisite for that work.